

Short Answer

The better long-term architecture is:

Keep Raw Data as the complete, unedited/audit source, then create downstream output sheets that contain only the rows and columns each workflow actually needs.

But I would not blindly copy spec/Process Data (1).txt as-is. The best path is a hybrid:

Raw Data = immutable source of truth

no destructive edits

no deleted rows

no removed columns

ideally no added processing columns either

Processing layer = in-memory first, optional "Processed Data" sheet only if you want a visible audit/work layer

use all Raw Data rows to calculate contacts, phones, addresses, notes, banners, primary PMR ownership, disenrollment exclusion, etc.

only after those calculations are complete should non-primary/secondary rows be removed from output views

Demo P / Master List / Monthly Change / Disenrolled = governed outputs

mapped to Dashboard/template headers

trimmed to necessary columns

row-reduced according to business purpose

Master List should remain primary-row only

Demo P should either be primary-row only after contact/secondary data is rolled up, or retain secondary rows only if a downstream workflow still depends on them

That gives you the audit safety of Raw Data, the performance of trimmed downstream sheets, and the governance alignment of Dashboard-owned templates.

Why I Would Not Keep "All Source Data Across All Sheets"

Keeping all source rows and all source columns across Raw Data, Demo P, Master List, Monthly Change, Disenrolled, etc. is the safer short-term behavior, but it is not the best architecture.

It creates these problems:

Performance drag: every workflow reads/writes more rows and columns than it needs.

Duplicate source-of-truth problem: if every sheet contains a copy of all raw source detail, it becomes unclear which sheet is authoritative.

Higher risk of accidental edits: downstream sheets become editable copies of source data instead of governed outputs.

Dashboard/template drift risk: governed sheets should follow Dashboard-defined headers and behavior, not preserve arbitrary import columns forever.

Harder validation: QA has to distinguish source columns, generated columns, hidden columns, working columns, and final columns on every output sheet.

The repository README explicitly says the framework should follow Read Once → Process In Memory → Write Once, and that runtime column additions/removals should be avoided in favor of dashboard/template governance.

What the Current src/Code.gs Process Does

Current Raw Data behavior

The current formatRawData() formats the active source sheet in place. It:

ensures Raw Data title/header row structure,

preserves current headers,

explicitly records “no column removal,”

populates approved sync columns,

renames the sheet,

locks row heights.

Key point: current Raw Data is not fully unedited. It is preserved in terms of source rows/headers, but the workflow still formats it and appends approved sync columns like Primary PMR Row and Banner fields.

Current Demo P behavior

processDemoP() can build a Demo P sheet from Raw Data. The notification explicitly says Raw Data is preserved when Raw Data is the source, Demo P contains active participants only, and Master List can be created from primary PMR rows only.

createActiveDemoPFromRawData_() reads Raw Data, splits active vs disenrolled rows, maps active rows to Demo P dashboard headers, creates a Demo P sheet from the dashboard template, and then runs working-source processing on that Demo P sheet.

Important: the active Demo P output is built from `split.activeRows`, which includes all active rows in a PMR block, not only the primary row. The `split` function pushes every row in an active PMR block into `activeRows`.

Current Master List behavior

The current Master List already follows the lean-output direction. `createMasterList()` copies from processed Demo P into the Master List, then right-sizes the output.

`copyPrimaryDemoPRowsToMasterListByHeader_()` first tries to include only rows where Primary PMR Row is true. If no primary rows are found, it falls back to one row per PMR using DOB or first-row logic.

So the current architecture is already partially hybrid:

Raw Data

= mostly preserved but formatted/enriched

Demo P

= active participants only, but may retain secondary rows

Master List

= primary rows only

Disenrolled

= primary/first row per disenrolled PMR block

What spec/Process Data (1).txt Does Differently

It introduces a Processed Data output

In `spec/Process Data (1).txt`, `formatRawData()` creates a separate Processed Data sheet from a template, copies Raw Data values into that template, prunes rows, runs processing, and finalizes the Processed Data output. {line_range_start=2776 line_range_end=2823 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2776-L2823"}

That is closer to the architecture you are considering:

Raw Data source

↓

Processed Data working/output sheet

↓

Pruned/processed downstream sheets

It maps Raw Data into governed Processed Data headers

`copyRawDataToProcessedTemplate_()` reads the Raw Data headers, reads the Processed Data template headers, maps source columns into template columns by normalized header name, clears the Processed Data data area, and writes the mapped rows into Processed Data. {line_range_start=2905 line_range_end=2948 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2905-L2948"}

That is a strong pattern because it prevents arbitrary source columns from leaking into downstream outputs.

It prunes non-primary rows from Processed Data

prepareProcessedDataPrimaryRowsAndPrune_() processes contacts first, marks primary PMR rows, deletes non-primary rows, moves/prunes disenrolled rows, and returns counts for primary rows kept and rows deleted. {line_range_start=2974 line_range_end=2999 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L2999"}

deleteNonPrimaryProcessedRows_() deletes rows where Primary PMR Row = No.
{line_range_start=3001 line_range_end=3022 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L3001-L3022"}

That is the key architectural difference: secondary rows are used for processing, then removed from the processed output.

It also states the Raw Data source-only principle

The Process Data script's Raw Data → Demo P builder explicitly documents that Raw Data remains source-only, no processing columns are added to Raw Data, and active rows are processed only after disenrolled PMRs are excluded. {line_range_start=3909 line_range_end=3917 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L3909-L3917"}

That principle is better than modifying Raw Data with processing columns.

Important Caveat About Process Data (1).txt

Process Data (1).txt is not perfectly clean or perfectly aligned with current governance as-is.

For example:

Its header says it is v1.4.21.5, while the current src/Code.gs is a later framework state.
{line_range_start=1 line_range_end=16 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L1-L16"}

Its processDemoPAsWorkingSource_() still assigns primary PMR rows and syncs Banner source into Demo P memory, while the current script explicitly says Primary PMR is assigned during Format Raw Data and should not be reassigned in Demo P. {line_range_start=3650 line_range_end=3685 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L3650-L3685"}

Its createActiveDemoPFromRawData_() still maps all active Raw Data rows into Demo P, not only primary rows. {line_range_start=3938 line_range_end=3981 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L3938-L3981"}

The current script's governance comments say runtime column insertion/deletion is retired and Dashboard Section C is the column authority.

So I would use Process Data (1).txt as an architecture reference, not as a direct replacement.

Recommended Architecture

Best model

1. Raw Data

- complete source import
- do not delete rows

- do not delete columns
- preferably do not add processing columns
- may optionally create a formatted Raw Data VIEW, but keep original import untouched

2. Processing Layer

- read Raw Data once
- calculate in memory:
 - Primary PMR Row
 - contact rollups
 - phone splits
 - address rollups
 - notes summary
 - banner summary
 - disenrollment classification
- optionally write a Processed Data sheet if you want a visible audit/work product

3. Demo P

- dashboard/template governed columns only
- active participants only
- either:
 - A. primary rows only after contact rollups, or
 - B. all active rows only if downstream workflows still require secondary/contact row-level data

4. Master List

- primary rows only
- dashboard/template governed columns only

5. Monthly Change

- generated from current vs previous governed participant outputs
- should not require full raw secondary rows if Demo P/Master List already carry rolled-up fields

6. Disenrolled

- one primary/first row per disenrolled PMR block
- enough columns to audit why excluded

My Recommendation by Sheet

Sheet	Keep All Source Rows?	Keep All Source Columns?	Recommended Direction
Raw Data	Yes	Yes	Preserve complete import as audit source.
Processed Data, if retained	No	No	Use all rows for processing, then primary-row output only.
Demo P	Maybe temporarily	No	Prefer primary-row only after contact/secondary rollups are proven complete.
Master List	No	No	Primary-row only. Current code already follows this direction.
Monthly Change	No	No	Section-based output only.
Disenrolled	No	No	One row per disenrolled PMR.
QA / Timing / Logs	No	No	Summary/detail rows only.

Decision

I would choose the Process Data style approach, with current-governance updates.

Specifically:

Raw Data should be the complete immutable source, and downstream sheets should be minimized to the data each sheet actually owns.

That is better than retaining all source data across every sheet.

But I would implement it in phases:

Suggested Phased Migration

Phase 1 — Stop treating Raw Data as a working sheet

Keep Raw Data complete and source-only.

Current `formatRawData()` still formats and enriches the active Raw Data sheet with approved sync columns.

I would change the target design so Raw Data is either:

completely untouched source import, or

source import plus a separate dashboard-formatted Raw Data output/view.

Do not make Raw Data the place where processing columns are added.

Phase 2 — Add or formalize a processing layer

Use the Process Data pattern:

copy/mapping from Raw Data into dashboard-governed headers,

process in memory,

```
prune non-primary rows only after secondary/contact data has been rolled up. {line_range_start=2905
line_range_end=2948 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data
(1).txt#L2905-L2948"}{line_range_start=2974 line_range_end=3022 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L3022"}
```

This could be either:

a visible Processed Data MM.yy sheet, or

an internal in-memory object with optional debug output.

Given the framework's one-pass principle, I prefer in-memory first, with a visible Processed Data sheet only if you want audit transparency.

Phase 3 — Make Demo P primary-row only, but only after validation

Current Demo P can still carry all active rows because `splitRawDataRowsIntoActiveAndDisenrolled_()` pushes every row in active PMR blocks into `activeRows`.

Before trimming Demo P to primary-row only, confirm these are fully rolled up:

contacts,

phone labels/values,

addresses,

notes,

language fields,

banner summary,

any Monthly Change comparison fields.

If those are all represented on the primary row, Demo P should become primary-row only.

Phase 4 — Keep Master List primary-only

The current Master List workflow is already correct directionally: it copies primary Demo P rows and writes only those rows to Master List.

I would preserve this.

Risk Assessment

Keeping all source data across all sheets

Pros

Low risk short-term.

Easier to debug because every output sheet still has source context.

Less chance of breaking workflows that secretly rely on secondary rows.

Cons

Slower.

Larger sheets.

More duplicate data.

Harder governance.

More places for users to edit copied source data.

Weak source-of-truth boundary.

Raw Data immutable + trimmed downstream sheets

Pros

Cleaner architecture.

Faster processing.

Stronger audit trail.

Less sheet bloat.

Better Dashboard/template alignment.

Clear source vs output ownership.

Easier QA once stabilized.

Cons

Requires careful migration.

Must guarantee all secondary-row data is rolled up before deletion.

Monthly Change/contact workflows may need adjustment if they currently depend on secondary rows.

More up-front testing required.

Final Recommendation

Use this rule:

Keep complete data only once: Raw Data. Everywhere else should be purpose-built, governed, and as small as possible.

So yes, the Process Data-style approach is better, but I would not directly revert to the old Process Data (1).txt script. I would modernize that concept into the current framework:

Raw Data = immutable source

Processing = in memory / optional Processed Data

Demo P = active participant output, eventually primary-row only

Master List = primary-row only

Monthly Change = section report only

Disenrolled = one row per excluded PMR

That best matches the framework's documented principles: Dashboard Driven Architecture, Template First Formatting, Primary PMR Architecture, and One Pass Processing.

Commands Used

```
✓ find .. -name AGENTS.md -o -name AGENT.md | sort && sed -n '1,220p' AGENT.md && git fetch origin --prune && git checkout -B work origin/main && git status -sb && git log -1 --oneline && find . -maxdepth 2 -type f | sort | head -300
```


✓ `wc -l AGENT.md README.md 'spec/Process Data (1).txt' src/Code.gs`
Current_Production/v1.5.55_Current_Production_Script && sed -n '1,260p' README.md && nl -ba 'spec/Process Data (1).txt' | sed -n '1,260p' && rg -n "function |const |RAW DATA|Raw Data|Demo P|Master List|processData|Process Data|secondary|column|delete|trim|remove" src/Code.gs | sed -n '1,260p'

✓ `rg -n "function (formatRawData|createRawData|processRaw|createActiveDemoP|processDemoP|createMasterList|build Demo|removeSecondary|hideNonPrimary|delete.*Secondary|delete.*Rows|trim|columns|keep|retain|copyRaw|create.*Output|splitRawData)" src/Code.gs 'spec/Process Data (1).txt' | sed -n '1,260p'`

✓ `nl -ba 'spec/Process Data (1).txt' | sed -n '2776,3035p;3556,4035p;4727,4795p;5450,5535p;7204,7225p;10979,11060p'`

✓ `nl -ba src/Code.gs | sed -n '3205,3378p;3385,3536p;3537,3605p;3606,3690p;3938,3988p;3989,4025p;5505,5545p;5546,5588p;5589,5625p'`

✓ `nl -ba 'spec/Process Data (1).txt' | sed -n '3897,4035p;4036,4075p;3650,3705p;5450,5535p;5536,5585p'`

No files were changed, so I did not create a commit or PR.

File (1)

The current architecture copies existing unchanged data to the following months to keep from reprocessing the entire data base every month. What if Processed Data was a up to date data source to be used for the Master List. and other sheets note yet started. So after initial creation each month only data that has changed would be replaced on Processed Data.

Copy

Short Answer

Yes — Processed Data could become the up-to-date working data source for Master List and future sheets, and that is probably the cleanest long-term architecture.

But I would define it very carefully:

Raw Data should remain the immutable monthly source input. Processed Data should become the normalized, governed, current participant-state layer. Master List and future outputs should be generated from Processed Data, not directly from Raw Data or Demo P.

That would let you keep the benefit of the current monthly incremental update model — copying existing unchanged data forward — while moving that logic one layer earlier, into a cleaner data source.

What You Are Proposing

Your idea is essentially this:

Raw Data = monthly import / source evidence

Processed Data = current normalized participant data store

Master List = governed output generated from Processed Data

Other future sheets = generated from Processed Data

Then each month:

Previous Processed Data

↓ copy forward

Current Raw Data / Monthly Change

↓ detect changed PMRs

Replace only changed PMRs in Processed Data

↓

Generate Master List and other outputs

That is a strong architecture.

It is better than making Master List itself the carried-forward data store, because Master List is really an output/reporting product. Processed Data is a better place to hold normalized operational state.

How This Compares To Current Architecture

Current architecture

Right now, the monthly update flow creates the Monthly Change report, then updates the Master List.

The high-level runMonthlyUpdate() function builds Monthly Change first, then calls updateMasterListForMonth_().

updateMasterListForMonth_() currently:

Finds current Demo P.

Finds previous Demo P.

Finds previous Master List.

Copies previous Master List to the current month.

Gets or builds Monthly Change.

Rebuilds changed PMRs on the copied Master List.

Applies final Master List processing/display steps.

That means the current carried-forward state is effectively:

Previous Master List → Current Master List

The changed PMRs are read from the Monthly Change report.

rebuildChangedPMRsOnExistingMaster_() reads changed PMRs, updates/removes/adds affected PMRs, then processes only affected PMRs when possible.

The action function splits Monthly Change PMRs by section:

enrollments,

disenrollments,

demographic/caseload/banner updates,

contact changes.

Your proposed architecture

You are suggesting:

Previous Processed Data → Current Processed Data

Current Processed Data → Master List

That is cleaner because Processed Data becomes the carried-forward state, while Master List becomes a regenerated downstream output.

Why Processed Data Is a Better “Current State” Layer Than Master List

1. Master List is an output, not the best canonical data store

Current createMasterList() already treats Master List as an output created from another sheet. It locates the current Demo P, copies a Master List template, builds headers, copies primary Demo P rows, right-sizes rows, fills metadata, and applies final display.

The copy function copyPrimaryDemoPRowsToMasterListByHeader_() is explicitly primary-row oriented. It tries to copy rows where Primary PMR Row is true, then falls back to DOB or first-row logic only if no primary rows are detected.

That means Master List is already conceptually downstream of a more complete source.

2. Processed Data can preserve normalized business state without output formatting baggage

The old Process Data (1).txt already points in this direction. Its formatRawData() creates a separate Processed Data sheet from a template, maps Raw Data into it, prunes it, runs workflow processes, and finalizes it. {line_range_start=2776 line_range_end=2823 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2776-L2823"}

copyRawDataToProcessedTemplate_() maps Raw Data into Processed Data by matching normalized headers from Raw Data to the Processed Data template headers. {line_range_start=2905 line_range_end=2948 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2905-L2948"}

Then prepareProcessedDataPrimaryRowsAndPrune_() performs contact processing, marks primary PMR rows, deletes non-primary rows, and handles disenrolled/no-DOB pruning. {line_range_start=2974 line_range_end=3022 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L3022"}

That is very close to the data-store architecture you are describing.

3. It fits the framework principles

The README says the framework should follow:

Read Once

↓

Process In Memory

↓

Write Once

and should avoid repeated spreadsheet reads/writes.

It also says the framework is Dashboard-driven, with templates controlling sheet structure and formatting.

A governed Processed Data layer fits those rules if it is:

template/dashboard controlled,

updated in PMR-level batches,

used as the source for downstream outputs,

not manually edited as a freeform data store.

Recommended Model

I would make Processed Data the canonical monthly/current participant-state layer, with these rules:

Raw Data

Purpose:

Monthly source evidence / import archive

Rules:

never delete source rows,

never delete source columns,

do not use it as a working sheet,

use it to rebuild Processed Data if needed,

keep enough naming/date metadata to trace source month.

Processed Data

Purpose:

Normalized current participant state

Rules:

one current row per active participant/PMR after secondary rows are rolled up,

dashboard-governed columns only,

includes derived fields like:

Primary PMR Row,

Banner Summary,

Notes Summary,

contact rollups,

phone fields,

address fields,

language fields,

update status,

update month,

source sheet,

source hash/checksum if possible,

last updated timestamp.

copied forward monthly,

patched only for affected PMRs,

source for Master List and future outputs.

Master List

Purpose:

Presentation/governed participant output

Rules:

generated from Processed Data,

no source-state ownership,

can be fully regenerated from Processed Data,

should not be the incremental state store.

Monthly Change

Purpose:

Delta detection / audit instruction sheet

Rules:

identify changed PMRs,

categorize change type,

feed Processed Data replacement/update logic,

feed Master List update/rebuild logic only downstream.

Best Monthly Flow

I would target this flow:

1. Import Raw Data MM.yy



2. Build or update Processed Data MM.yy

- copy previous Processed Data forward
- identify changed PMRs from Raw Data / Monthly Change
- replace entire PMR records for changed PMRs
- add enrollments
- remove disenrollments
- recompute derived fields for changed PMRs



3. Build Demo P / Master List / future outputs from Processed Data



4. Validate Dashboard Quality + Timing

More specifically:

Previous Processed Data



copy to Current Processed Data



Monthly Change PMR set



for each changed PMR:

- delete old PMR row from Current Processed Data
- rebuild that PMR from current Raw Data
- recompute contacts/phones/address/notes/banner
- write updated PMR row



for enrollments:

- add new PMR row



for disenrollments:

- remove PMR from Processed Data
- write one row to Disenrolled output



Master List reads Current Processed Data

Important Design Point: Replace Whole PMRs, Not Individual Cells

If you make Processed Data a current-state table, I would not patch individual changed cells directly.

Instead:

Use PMR-level replacement.

Why?

Because one changed source field can affect several derived fields.

Examples:

Contact row changes can affect contact rollup columns.

Address fields can affect combined address output.

Banner source changes can affect Banner Summary.

Enrollment/disenrollment status can affect whether a PMR should exist at all.

Notes changes can affect Notes Summary.

So the safe unit of update is:

One PMR participant record / primary row
not:

One field / one cell

The current Master List update already thinks in PMR sets. It reads changed PMRs from Monthly Change, then creates affected PMR filters and performs PMR-level update operations.

I would preserve that idea, but move it to Processed Data.

How This Would Change Current Update Logic

Current

Previous Master List

↓ copied to current Master List

Monthly Change

↓ changed PMRs

Current Demo P

↓ changed PMR data

Current Master List updated

This is visible in `updateMasterListForMonth_()`, which copies the previous Master List and then rebuilds changed PMRs on the copied Master List.

Proposed

Previous Processed Data

↓ copied to current Processed Data

Monthly Change / Raw Data

↓ changed PMRs

Current Processed Data updated

↓

Master List generated from Current Processed Data

Then createMasterList() becomes simpler because it no longer needs to rely on Demo P as the source. It would source from Processed Data instead of current Demo P. Today, createMasterList() explicitly locates current Demo P and copies primary Demo P rows.

What This Solves

1. Less reprocessing

You keep the current benefit:

Only changed PMRs are rebuilt.

But now that benefit applies at the data layer, not just the Master List output layer.

2. Cleaner downstream sheets

Once Processed Data is current, downstream sheets can be generated from it without carrying Raw Data's secondary rows or unneeded columns.

3. Better future scalability

You mentioned "other sheets not yet started." This is where the design becomes powerful:

Processed Data

- |— Master List
- |— Demo P output
- |— Monthly Change
- |— Disenrolled
- |— future dashboard sheets
- |— future audit sheets
- |— future reporting sheets

Each new output can read the same normalized current-state layer instead of rebuilding from Raw Data.

4. Better audit separation

Raw Data remains evidence.

Processed Data becomes state.

Master List becomes output.

That separation is much easier to validate.

Biggest Risks

Risk 1 — Stale derived data

If Processed Data is carried forward, any bug in a derived field can persist month after month.

Mitigation:

keep a full rebuild function:

Rebuild Processed Data from Raw Data

store source month/source sheet/source hash per PMR,

add Dashboard Quality checks comparing changed PMRs against Raw Data.

Risk 2 — Missed changes

If Monthly Change fails to detect a PMR, Processed Data will not update that PMR.

Mitigation:

calculate a per-PMR source hash from Raw Data,

compare current Raw PMR hash vs previous Processed Data source hash,

update PMR if hash changed even if Monthly Change missed it.

Risk 3 — Contact/secondary-row loss

If Raw Data secondary rows are removed too early, contact/phone/note rollups can be wrong.

Mitigation:

always rebuild a changed PMR from the full Raw Data PMR block,

only write the final rolled-up primary row to Processed Data after all secondary-row logic runs.

The Process Data script shows the right idea by processing ranked contacts before deleting non-primary rows. {line_range_start=2974 line_range_end=2982 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L2982"}

Risk 4 — Manual edits to Processed Data

If users manually edit Processed Data, it becomes a second source of truth.

Mitigation:

protect Processed Data,

mark it as framework-owned,

only allow controlled update workflows,

use Master List or dedicated correction workflows for approved edits.

Recommended Data Columns for Processed Data

If Processed Data becomes the current source for Master List and future sheets, I would add governed metadata columns such as:

Processed Data Update Status

Processed Data Update Month

Processed Data Source Raw Sheet

Processed Data Source PMR Hash

Processed Data Last Updated At

Processed Data Update Reason

Processed Data Source Workflow

These are not display/report fields. They are control/audit fields.

They let you answer:

Was this PMR carried forward?

Was this PMR rebuilt this month?

Which Raw Data sheet fed it?

Did source data change?

Which workflow updated it?

Can this row be trusted?

Recommended Algorithm

Initial month / full rebuild

1. Read Raw Data.
2. Group by PMR.
3. For each PMR:
 - identify primary row
 - roll up contacts
 - roll up phones
 - combine address fields
 - combine notes
 - sync banner/care-plan-derived fields as allowed
 - classify active vs disenrolled
4. Write active PMRs to Processed Data.
5. Write disenrolled PMRs to Disenrolled output.
6. Build Master List from Processed Data.

Later months / incremental update

1. Copy previous Processed Data to current Processed Data.
2. Read current Raw Data.
3. Read previous Processed Data metadata/hashes.
4. Build PMR change set from:
 - Monthly Change report
 - Raw PMR hash differences
 - enrollments
 - disenrollments
5. For each changed PMR:
 - delete current copied Processed Data row for PMR
 - rebuild PMR from current Raw Data block
 - write updated Processed Data row
6. For disenrolled PMRs:
 - delete from Processed Data
 - write to Disenrolled output
7. Generate Master List from Processed Data.

8. Run Dashboard Quality / Timing checks.

My Recommendation

I would move toward your idea.

The target architecture should be:

Raw Data = immutable monthly source

Processed Data = current normalized state

Master List = output from Processed Data

Other future sheets = outputs from Processed Data

Monthly Change = delta/audit driver

That is better than:

Raw Data → Demo P → Master List

Previous Master List → Current Master List

because it gives the framework a true governed data layer.

However, I would implement it incrementally:

Phase 1

Add Processed Data as a generated sheet from Raw Data, using the Process Data (1).txt concept.

Phase 2

Make Master List able to source from Processed Data instead of Demo P.

Phase 3

Add monthly carry-forward for Processed Data.

Phase 4

Patch only changed PMRs in Processed Data.

Phase 5

Make Demo P and other future sheets outputs of Processed Data, not separate working sources.

Final Decision

Yes — Processed Data as the current, up-to-date data source is the better long-term architecture, as long as:

Raw Data remains the immutable evidence layer,

Processed Data is protected and framework-owned,

PMRs are replaced as whole participant records,

all derived fields are recalculated for changed PMRs,

a full rebuild path remains available,

hashes/metadata prevent stale rows,

Master List becomes an output, not the carried-forward data store.

That architecture gives you the performance advantage of incremental monthly updates without making Master List carry the responsibility of being the system's source of truth.

Commands Used

```
✓ git status -sb && git log -1 --oneline --decorate && rg -n "function  
(runMonthlyUpdate|updateMasterListForMonth_|getOrBuildMonthlyChangeReport_|rebuildChangedPMR  
sOnExistingMaster_|getChangedPMRsFromMonthlyChangeReport_|copyPreviousMasterListToCurrentM  
onth_|createMasterList|copyPrimaryDemoPRowsToMasterListByHeader_|formatRawData|copyRawData  
ToProcessedTemplate_|prepareProcessedDataPrimaryRowsAndPrune_)" src/Code.gs 'spec/Process  
Data (1).txt' | sed -n '1,220p'
```

```
✓ nl -ba src/Code.gs | sed -n  
'323,354p;4009,4054p;4056,4118p;5216,5298p;5298,5355p;5380,5468p;5421,5488p;5538,5625p'
```

```
✓ nl -ba 'spec/Process Data (1).txt' | sed -n '2776,2823p;2905,2948p;2974,3022p'
```

```
✓ nl -ba src/Code.gs | sed -n '7297,7368p;8010,8105p;9370,9465p;9520,9585p'
```

No files were changed, so I did not create a commit or PR.

I think the Processed Data can replace Demo P

Copy

Short Answer

I agree: Processed Data can replace Demo P as the main working/source layer.

More precisely:

Processed Data should become the governed participant-state source. Demo P should either be retired or reduced to an optional compatibility/export view.

That is cleaner than continuing to use Demo P as the framework's working source, because "Demo P" is really an imported/report-source concept, while "Processed Data" better describes the normalized, framework-owned, updated participant dataset.

Why This Makes Sense

Right now, Demo P is doing too many jobs:

Demo P currently acts as:

1. Raw/Data-derived participant working sheet
2. active participant filter
3. contact/phone/address/notes processing target
4. source for Monthly Change
5. source for Master List
6. source for update/audit workflows

That is why the code has so many "Demo P" dependencies. For example, createMasterList() currently locates the current Demo P sheet and copies primary rows from processed Demo P into Master List.

The current monthly update flow also locates current Demo P, previous Demo P, and previous Master List before building Monthly Change and updating Master List.

If Processed Data becomes the canonical working layer, the architecture becomes clearer:

Raw Data = source import / audit evidence

Processed Data = normalized current participant state

Master List = output from Processed Data

Monthly Change = delta/audit report from Processed Data

Demo P = optional export/view, if still needed

The Key Architectural Change

Current model

Raw Data



Demo P



Master List

Current code reflects this. `createActiveDemoPFromRawData_()` reads Raw Data, excludes disenrolled PMRs, maps active rows into Demo P dashboard headers, creates a Demo P sheet, and processes that sheet as the working source.

Then Master List reads Demo P. `copyPrimaryDemoPRowsToMasterListByHeader_()` reads Demo P data and copies primary PMR rows into Master List.

Better model

Raw Data



Processed Data



Master List



Other outputs

The `spec/Process Data (1).txt` script already sketches this direction. It defines both `DEMO_P_PREFIX` and `PROCESSED_DATA_PREFIX`, and it has a `SHEET_TYPE.PROCESSED_DATA` separate from `SHEET_TYPE.DEMO_P`.
{line_range_start=22 line_range_end=38 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L22-L38"}
{line_range_start=71 line_range_end=82 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L71-L82"}

Its `formatRawData()` creates a Processed Data sheet from a template, copies Raw Data into it, prunes primary rows, runs workflow processing, and returns the Processed Data sheet.
{line_range_start=2776
line_range_end=2823 path=spec/Process Data (1).txt
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2776-L2823"}

That is the right conceptual direction.

What Processed Data Should Own

Processed Data should own all normalized participant-level fields needed downstream:

Identity / core participant fields

Participant name

Preferred name

DOB

PMR

enrollment/capitation dates

enrollment status

disenrollment dates/reason

Rolled-up participant fields

address rollups

phone rollups

language fields

notes summary

contact summary / ranked contact fields

banner summary

caseload fields

Framework fields

Primary PMR Row

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Processed Data Source Hash

Processed Data Last Updated At

Governance fields

source month

source workflow

source raw row/block reference if needed

last rebuilt reason

That lets Master List become a clean output from Processed Data instead of a partially stateful monthly carry-forward artifact.

What Demo P Becomes

I would not keep Demo P as the internal source layer.

Instead, choose one of these:

Option A — Retire Demo P internally

Best long-term.

Process Demo P menu item → renamed to Build Processed Data

Demo P functions → migrated to Processed Data functions

Master List reads Processed Data

Monthly Change compares Processed Data

Option B — Keep Demo P as a compatibility output

Safer short-term.

Processed Data = framework source

Demo P = optional generated view/export

This allows any user-facing expectations around a “Demo P” sheet to continue, while the framework itself moves to Processed Data.

I recommend Option B first, then retire Demo P later after validation.

Why Processed Data Is a Better Name and Layer

“Demo P” sounds like an external/source report.

“Processed Data” sounds like:

framework-owned,

normalized,

validated,

up-to-date,

safe for downstream generation.

That matches what you want it to do.

The old Process Data script’s Raw Data → Processed Data path maps Raw Data into Processed Data template headers, instead of letting arbitrary Raw Data columns drive downstream outputs.

```
{line_range_start=2905 line_range_end=2948 path=spec/Process Data (1).txt  
git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2905-L2948"}
```

It also processes contact data before deleting non-primary rows, then prunes non-primary rows by the Primary PMR Row column. {line_range_start=2974 line_range_end=3022 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L3022"}

That is exactly the kind of behavior a “Processed Data” source layer should have.

What Must Change If Processed Data Replaces Demo P

This is not just a rename. Several workflows currently depend on “Demo P” semantics.

1. Master List creation must source from Processed Data

Current:

createMasterList

→ getCurrentDemoPSheet_

→ copyPrimaryDemoPRowsToMasterListByHeader_

The current function explicitly requires Demo P and tells the user to run Process Demo P if Demo P is missing.

Future:

createMasterList

→ getCurrentProcessedDataSheet_

→ copyProcessedDataRowsToMasterListByHeader_

Master List should no longer care whether data came from Demo P. It should only care that Processed Data is current and validated.

2. Monthly Change must compare Processed Data instead of Demo P

Current getOrBuildMonthlyChangeReport_() requires current Demo P and previous Demo P, formats date columns on both, and compares them using compareRawDemoPForSectionReport_().

Future:

getOrBuildMonthlyChangeReport_

→ currentProcessedData

→ previousProcessedData

→ compareProcessedDataForSectionReport_

The comparison fields can remain mostly the same, but the function names and source sheet logic should shift away from Demo P.

3. Monthly update must copy previous Processed Data, not previous Master List

Current monthly update copies previous Master List forward, then applies changed PMRs to that copied Master List.

Future:

Previous Processed Data

↓ copy forward

Current Processed Data

↓ apply PMR-level deltas

Master List generated from Current Processed Data

That means Master List is no longer the thing being carried forward. Processed Data is.

4. Update actions must update Processed Data first

Current `updateMasterListFromMonthlyChangeActions_()` applies Monthly Change actions directly to Master List using current Demo P as the source for additions/updates/contact merges.

Future:

`updateProcessedDataFromMonthlyChangeActions_`

→ remove disenrolled PMRs from Processed Data

→ add enrollment PMRs from Raw Data / Processed Data builder

→ replace primary participant records

→ replace contact rollups

→ mark update metadata

then:

`buildMasterListFromProcessedData_`

This prevents Master List from becoming the state mutation layer.

Recommended Future Flow

Initial month

Raw Data

↓

Build Processed Data

- map Raw Data to governed Processed Data headers
- process contacts/phones/address/language/notes/banner
- assign Primary PMR Row
- remove secondary rows after rollups
- remove or export disenrolled PMRs

↓

Build Master List from Processed Data

Later months

Previous Processed Data

↓ copy forward

Current Processed Data

↓ patch only changed PMRs

Current Raw Data / Monthly Change

↓ source for changed PMR replacement

Current Processed Data

↓

Master List

↓

Other outputs

What I Would Rename

To make the architecture clear, I would eventually rename concepts like this:

Current Name Future Name

processDemoP buildProcessedData or processRawDataToProcessedData

getCurrentDemoPSheet_ getCurrentProcessedDataSheet_

getPreviousDemoPSheet_ getPreviousProcessedDataSheet_

compareRawDemoPForSectionReport_ compareProcessedDataForSectionReport_

copyPrimaryDemoPRowsToMasterListByHeader_ copyProcessedDataRowsToMasterListByHeader_

updatePrimaryRowsFromDemoPForPMRs_ updateProcessedRowsForPMRs_

mergeSecondaryRowsFromDemoPForPMRs_ maybe removed if Processed Data is primary-row
with contact rollups

I would keep aliases temporarily so existing menus/buttons do not break immediately.

Critical Design Rule

If Processed Data replaces Demo P, then Processed Data must not simply be "Demo P with a new name."

It must be:

Raw Data full block

↓

full PMR processing

↓

one normalized participant record per active PMR

That means secondary/contact rows must be consumed before they are removed.

The Process Data script already does this in the right order: process ranked contacts first, then mark primary rows, then delete non-primary rows. {line_range_start=2974 line_range_end=3022 path=spec/Process Data (1).txt git_url="https://github.com/tabs912/ML/blob/main/spec/Process Data (1).txt#L2974-L3022"}

That sequence is important.

What To Preserve From Demo P

Even if Demo P goes away as a working source, the logic currently attached to Demo P should not be thrown away.

Preserve the logic for:

active/disenrolled splitting,

primary PMR assignment,

banner summary,

address combining,

language processing,

phone splitting,

contact processing,

notes summary,

Monthly Change comparison fields,

Master List mapping.

Move those behaviors to the Processed Data builder.

Current processDemoPAsWorkingSource_() is where many of those transformations happen today. It updates banner summary, combines address fields, processes language, splits phone fields, processes contacts, combines notes, and writes processed data back to the sheet.

Those are Processed Data responsibilities, not necessarily Demo P responsibilities.

My Recommendation

Yes — make Processed Data replace Demo P as the framework's internal working source.

But do it in phases:

Phase 1 — Introduce Processed Data as a parallel output
Keep Demo P working for now.

Build Processed Data from Raw Data.

Validate row counts, PMRs, and output fields against Demo P/Master List.

Phase 2 — Make Master List read from Processed Data
Change Master List source from Demo P to Processed Data.

Keep Demo P as an optional export/view.

Phase 3 — Make Monthly Change compare Processed Data
Replace current/previous Demo P comparison with current/previous Processed Data comparison.

Keep field groups the same initially.

Phase 4 — Move incremental update state to Processed Data
Copy previous Processed Data forward.

Patch changed PMRs in Processed Data.

Regenerate Master List from Processed Data.

Phase 5 — Retire Demo P as required workflow
Remove Demo P as a required source.

Keep a compatibility menu item only if users still need a Demo P-formatted export.

Final Position

I agree with your direction.

The better architecture is:

Raw Data = original monthly input

Processed Data = canonical normalized framework data

Master List = output

Monthly Change = delta/audit report

Demo P = optional compatibility/export view

That makes the framework cleaner, faster, and easier to extend.

The main caution is that Processed Data must own all the transformations currently attached to Demo P before Demo P is removed as a source. If that migration is done carefully, Processed Data can fully replace Demo P.

Commands Used

```
✓ git status -sb && git log -1 --oneline --decorate && rg -n "Demo P|DEMO_P|Processed  
Data|PROCESSED_DATA|getCurrentDemoPSheet_|createActiveDemoPFromRawData_|processDemoP  
|createMasterList\\(|copyPrimaryDemoPRowsToMasterListByHeader_|compareRawDemoPForSectionRe  
port_|updateMasterListFromMonthlyChangeActions_" README.md src/Code.gs 'spec/Process Data  
(1).txt' | sed -n '1,260p'
```

```
✓ nl -ba src/Code.gs | sed -n  
'101,112p;3205,3286p;3385,3544p;3616,3680p;3806,3869p;5421,5488p;5538,5625p;6990,7068p;7297,  
7339p;8048,8088p;9370,9433p'
```

```
✓ nl -ba 'spec/Process Data (1).txt' | sed -n '22,38p;71,82p;2776,2823p;2905,2948p;2974,3022p'
```

No files were changed, so I did not create a commit or PR.

I don't think I would want a monthly Processed Data sheet, just making it ongoing. Add the columns like Master List Update Status, to note when the information was last updated. Master List Update Month, Master List Source Sheet. And could get rid of the Monthly Change Log by adding a columns updated column or I probably wouldn't need it.

Copy

Short Answer

Yes — I think your revised idea is even better:

Use one ongoing Processed Data sheet as the framework's current participant-state table, not monthly Processed Data MM.yy sheets.

That would make the architecture much cleaner:

Raw Data monthly imports = source evidence

Processed Data = current up-to-date participant state

Master List = output generated from Processed Data

Monthly Change = optional audit/report, not required state storage

Master List Change Log = optional, possibly replaceable by Processed Data metadata

I would not call the metadata columns Master List Update Status, Master List Update Month, and Master List Source Sheet inside Processed Data, though. I would generalize them:

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Processed Data Columns Updated

Processed Data Last Updated At

Processed Data Source PMR Hash

You can still map those into Master List metadata later if Master List needs its own output-tracking fields.

Why One Ongoing Processed Data Sheet Is Better Than Monthly Processed Data Sheets

A monthly Processed Data sheet would still create this pattern:

Processed Data 05.26

Processed Data 06.26

Processed Data 07.26

...

That keeps monthly history, but it also creates another monthly-sheet management problem.

Your revised idea is cleaner:

Processed Data

One living sheet.

It is always the current state of all active participants.

Then Raw Data sheets remain the historical monthly evidence.

That gives you:

less sheet clutter,

less copying,

easier source of truth,

easier Master List generation,

easier future sheet generation,

less need to carry full historical state forward every month.

Recommended Architecture

Raw Data MM.yy

= monthly source import / evidence

Processed Data

= single ongoing current participant state

Master List MM.yy

= monthly governed output generated from Processed Data

Monthly Change MM.yy

= optional audit/delta report

Disenrolled

= exclusion/history output for removed participants

Dashboard Quality / Timing

= validation and runtime reporting

What Processed Data Becomes

Processed Data becomes the framework's current participant database.

It should contain one current normalized participant row per active PMR, after all processing is done:

Primary PMR assignment,

contact rollups,

phone parsing,

address combining,

notes summary,

banner summary,

caseload fields,

enrollment/disenrollment state,

update metadata.

The README already says every participant has exactly one Primary PMR Row, that assignment happens during Raw Data processing only, and downstream synchronization targets only the Primary PMR Row.

That fits your idea perfectly. Processed Data should be the sheet that holds those current primary PMR rows.

Existing Code Already Has the Metadata Pattern

The framework already has metadata fields for Master List:

Master List Update Status

Master List Update Month

Master List Source Sheet

It also already hides those Master List metadata fields as non-display/report columns.

And fillMasterListMetadataColumns_() already writes:

update status,

update month,

source sheet.

So the pattern already exists.

I would reuse the concept, but rename it for Processed Data:

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Do not use Master List names for Processed Data, because Processed Data is upstream of Master List.

Suggested Processed Data Metadata Columns

I would add these to the governed Processed Data header set:

Column	Purpose
Processed Data Update Status	Created, Updated, Carried Forward, Rebuilt, Disenrolled, Error, etc.
Processed Data Update Month	The report month that last changed this participant.
Processed Data Source Sheet	Raw Data sheet or workflow source that last updated this row.
Processed Data Columns Updated	Comma-separated changed fields from the latest update.
Processed Data Last Updated At	Timestamp of the framework update.
Processed Data Last Updated By Workflow	Example: Format Raw Data, Monthly Update, Manual Rebuild.
Processed Data Source PMR Hash	Hash/checksum of the Raw Data PMR block used to detect future changes.
Processed Data Previous Source PMR Hash	Optional, useful for drift checks.
Processed Data Update Notes	Optional human-readable action details.

Minimum viable set:

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Processed Data Columns Updated

Processed Data Last Updated At

Processed Data Source PMR Hash

About Getting Rid of the Monthly Change Log / Change Log

I think you can reduce the need for a separate change log, but I would be careful about completely removing audit history.

There are two separate concepts here:

1. Monthly Change Report

This currently has a Columns With Change field. The code adds that header if missing.

When building a Monthly Change row, it writes the sorted list of changed columns into Columns With Change.

That same idea can move directly into Processed Data as:

Processed Data Columns Updated

2. Master List Change Log

The current Master List update logic logs deleted PMR blocks before deleting them. It captures timestamp, month, PMR, reason, source sheet, row number, report sheet, and serialized row data, then appends that to the Master List Change Log.

That is more detailed than a single Columns Updated column.

If you remove this log, you lose:

prior row snapshot,

deletion reason,

row-level deletion history,

structured audit trail for removed rows.

So I would not remove it immediately. I would first decide how much audit history you actually need.

Recommended Change-Tracking Model

If you want simple operations and do not need deep audit history

Then yes, you can likely replace most log usage with Processed Data metadata:

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Processed Data Columns Updated

Processed Data Last Updated At

This gives you current-state visibility but not full history.

If you need historical audit

Keep a small hidden system sheet, but make it generic:

Processed Data Change History

Instead of:

Master List Change Log

It could log only meaningful events:

PMR added,

PMR updated,

PMR disenrolled,

PMR restored,

source hash changed,

rebuild performed.

That would be much smaller than logging every Master List row operation.

Best Compromise

I would do this:

Keep on Processed Data

Processed Data Update Status

Processed Data Update Month

Processed Data Source Sheet

Processed Data Columns Updated

Processed Data Last Updated At

Processed Data Source PMR Hash

Keep an optional hidden lightweight history sheet

Processed Data Change History

Only write to it when:

a PMR is added,

a PMR is updated,

a PMR is removed/disenrolled,

a PMR hash changes unexpectedly,

a full rebuild happens.

Then the current Master List Change Log can eventually be retired because Master List is no longer the mutation layer.

How Monthly Update Would Work With One Ongoing Processed Data Sheet

Step 1 — Import Raw Data for the month

Raw Data 06.26

Raw Data stays as evidence.

Step 2 — Read current Processed Data

Processed Data

This is the current participant-state table.

Step 3 — Build PMR deltas

For each PMR in Raw Data:

compute a PMR-level source hash,

compare to Processed Data Source PMR Hash,

if hash changed, rebuild that PMR,

if PMR is new, add it,

if PMR is disenrolled, remove from active Processed Data and write to Disenrolled.

Step 4 — Update only changed PMRs

For each changed PMR:

delete old Processed Data row

rebuild from Raw Data PMR block

write updated row

set metadata columns

Step 5 — Generate Master List from Processed Data

Master List becomes an output only.

No need to carry Master List forward as the source of current participant state.

What Happens To Monthly Change?

You have two options.

Option A — Monthly Change remains as a report only

The framework can still create:

Monthly Change 06.26

But it becomes a report generated from update results, not the required source for updating Master List.

Option B — Remove Monthly Change as a required workflow

Processed Data metadata becomes the lightweight change record.

You may still keep Dashboard Quality validation for change counts.

I would choose Option A first, then decide later whether Monthly Change is still worth keeping.

Recommended Status Values

For Processed Data Update Status, I would use controlled values:

Created
Updated
Carried Forward
Rebuilt
Disenrolled
Restored
Source Hash Changed
No Change
Error
Example:

PMR	Status	Month	Source Sheet	Columns Updated
12345	Updated	06.26	Raw Data	06.26 Address 1 - Street, Phone 1 - Value
67890	Carried Forward	05.26	Raw Data	05.26
99999	Created	06.26	Raw Data	06.26 New PMR
55555	Disenrolled	06.26	Raw Data	06.26 Enrollment Status, Disenrollment Date

Why "Columns Updated" Is Useful But Not Enough Alone
Columns Updated is very useful for visibility.

But by itself, it does not tell you:

previous values,

row deletion history,

who/what changed it,

whether a row was rebuilt due to hash drift,

whether a row was removed because of disenrollment.

So I would pair it with:

Processed Data Update Status
Processed Data Last Updated At
Processed Data Source PMR Hash
That makes it much more reliable.

Revised Target Architecture
Raw Data 06.26

↓

Process Raw Data against existing Processed Data

↓

Processed Data

- one ongoing current-state sheet
- active participants only
- one primary row per PMR
- metadata says when/why/source/columns updated

↓

Master List 06.26

- generated output
- no state ownership

↓

Optional reports:

- Monthly Change 06.26
- Disenrolled 06.26
- Processed Data Change History

My Recommendation

I agree with you:

Do not create monthly Processed Data sheets. Make Processed Data ongoing.

And yes:

Processed Data can replace Demo P as the source for Master List and future outputs.

But I would not fully remove change logging until Processed Data has enough metadata and validation.

Recommended path:

Create one ongoing Processed Data sheet.

Add Processed Data metadata columns.

Update only changed PMRs each month.

Generate Master List from Processed Data.

Keep Monthly Change as optional/report-only at first.

Keep Master List Change Log temporarily.

After validation, replace Master List Change Log with lightweight Processed Data metadata and/or a smaller Processed Data Change History sheet.

Commands Used

```
✓ git status -sb && git log -1 --oneline --decorate && rg -n "Master List Update Status|Master List Update Month|Master List Source Sheet|Columns With Change|Master List Change Log|appendMasterListChangeLog_|fillMasterListMetadataColumns_|getMonthlyChangeReportHeaders_|buildMonthlyChangeReportRow_|Monthly Change|Processed Data Update|Source Sheet" src/Code.gs README.md 'spec/Process Data (1).txt' | sed -n '1,260p'
```

```
✓ nl -ba src/Code.gs | sed -n '245,260p;274,318p;1021,1032p;1103,1122p;5486,5492p;5788,5845p;8106,8138p;9200,9238p'
```

```
✓ nl -ba README.md | sed -n '82,91p;131,148p'
```

No files were changed, so I did not create a commit or PR.

The Disenrolled List could also be ongoing. Store all disenrolled participants there and current participant in Processed Data -- will keep the name Demo P for Processed Data

Copy

Short answer

Yes — that architecture makes sense:

Keep Raw Data as the imported/source snapshot.

Keep Demo P as the ongoing, current-state active participant source — even if conceptually it is “Processed Data,” retaining the name Demo P avoids breaking all current references.

Keep Disenrolled Exclusion List as the ongoing, current-state inactive/disenrolled participant source.

Make Master List read from Demo P for current participants and the Disenrolled Exclusion List for removals/exclusions, instead of relying on month-by-month Demo P comparisons forever.

That gives you a cleaner two-ledger model:

Sheet	Role	Contents	Monthly behavior
Raw Data		Source import / audit snapshot	Full imported file
		imported/formatted source	Preserve as
Demo P		Current active participant state	Active participants only
		add new active PMRs, remove newly disenrolled PMRs	Patch changed PMRs,
Disenrolled Exclusion List		Current disenrolled participant state	All known disenrolled PMRs
		Append/refresh newly disenrolled PMRs; do not recreate monthly unless needed	
Master List		Operational output	Primary PMR rows + downstream synced fields
		from Demo P and exclusion state	Rebuild/update

The existing code already points in this direction because Raw Data processing owns Primary PMR assignment, Demo P is already treated as the current processed working source, and the Disenrolled Exclusion List is already a framework-owned system sheet.

Why an ongoing Disenrolled List is the right companion to ongoing Demo P

If Demo P becomes the ongoing active participant ledger, then an ongoing Disenrolled Exclusion List is the natural inactive ledger.

Right now, the current script splits Raw Data into active and disenrolled PMR blocks by looking at Enrollment Status, disenrollment/date-of-death fields, or an external disenrolled set. Active rows go to activeRows; disenrolled PMRs go to disenrolledRows and disenrolledRowsByPMR.

That means the split already exists conceptually:

Raw Data

└─ active PMRs → Demo P
└─ disenrolled PMRs → Disenrolled Exclusion List

The current implementation still creates a dated disenrolled output name and deletes/replaces prior dated/legacy sheets during the append/write path. For your proposed architecture, that should change: the target should be a stable ongoing sheet named Disenrolled Exclusion List, not a new monthly disenrolled output.

Keep the name Demo P — but redefine it internally
I agree with keeping the name Demo P.

The current code is deeply built around Demo P as the working source. For example, createMasterList() locates the current processed Demo P sheet, copies primary PMR rows from Demo P, and fills Master List metadata using the Demo P sheet name as the source.

Likewise, monthly Master List update currently looks for current Demo P, previous Demo P, and previous Master List, then builds a Monthly Change Report from Demo P comparisons.

So instead of renaming all of that to Processed Data, I would do this:

Keep the user-facing sheet name as Demo P, but redefine Demo P as the ongoing processed/current active participant source.

That avoids a broad rename and preserves compatibility with existing functions, template headers, dashboard labels, and mental model.

Recommended new data flow

1. Format Raw Data

Raw Data should remain a source snapshot. The current script already says it preserves Raw Data headers, appends only approved sync columns, populates Primary PMR Row + banner fields, and tells the user to run Process Demo P next.

That is good. I would keep that principle.

2. Update ongoing Demo P

Instead of creating a new monthly Demo P sheet every time, Process Demo P should update the stable Demo P ledger:

For each PMR block in latest Raw Data:

if disenrolled:
 remove/expire from Demo P
 upsert into Disenrolled Exclusion List
else:
 upsert into Demo P
 set Demo P update metadata

The current code already has Demo P metadata columns available: Demo P Update Status, Demo P Update Month, and Demo P Source Sheet.

The current Demo P build path also already populates those update columns in memory when creating Demo P from Raw Data.

For the ongoing model, those columns become more important:

Column	Meaning
Demo P Update Status	Created, Updated, Unchanged, Removed to Disenrolled, Reinstated
Demo P Update Month	Most recent month this participant changed or was confirmed

Demo P Source Sheet	Raw Data sheet or workflow source that last updated the participant
Demo P Columns Updated	Comma-separated list of fields changed in the most recent update
Demo P Last Updated At	Exact timestamp of update
Demo P Source PMR Hash	Hash of latest PMR block used to detect changes

I would add Demo P Columns Updated, Demo P Last Updated At, and Demo P Source PMR Hash eventually, but the existing three metadata columns are already a good start.

3. Update ongoing Disenrolled Exclusion List

For disenrolled participants:

If PMR not already in Disenrolled Exclusion List:

- append primary/summary row
- status = Newly Disenrolled
- month = current month
- source = Raw Data sheet

If PMR already exists:

- update row if source fields changed
- status = Updated
- month = current month
- source = Raw Data sheet

The existing split stores only the primary/first row for a disenrolled PMR block. That is probably fine if the Disenrolled List is only an exclusion/retention ledger. If you want it to be a complete historical source, then it should store either:

the full disenrolled PMR block, or

one row plus enough summary/audit columns to understand why the PMR was excluded.

For your likely use case, I would keep one row per PMR, because the purpose is to prevent re-adding disenrolled participants and to explain removals.

4. Master List reads active source + exclusion source

Master List should become simpler:

Master List active source = Demo P

Master List exclusion source = Disenrolled Exclusion List

The existing Master List already has metadata columns for update status/month/source.

So the Master List can inherit provenance from Demo P:

Master List field	Source
Master List Update Status	Created/Updated/Unchanged/Rebuilt from Demo P
Master List Update Month	Month of latest Demo P update or Master List rebuild
Master List Source Sheet	Demo P
participant active/disenrolled state	Demo P + Disenrolled Exclusion List

This is better than relying forever on previous-month Demo P sheets.

What this would replace

It can reduce the need for monthly Demo P sheets

Currently, Monthly Change depends on current and previous Demo P sheets.

If Demo P becomes ongoing, you no longer need separate monthly Demo P sheets as the normal source of truth. You can still archive snapshots if you want, but the operational source becomes one stable Demo P.

It can reduce the need for Monthly Change Report as a required dependency

Current monthly update builds a Monthly Change Report first, then updates the Master List.

With ongoing Demo P + ongoing Disenrolled List, the Monthly Change Report can become optional/reporting-only because each Demo P row can say what changed, when it changed, and from what source.

However, I would not remove Monthly Change immediately. I would first make it optional:

Phase 1: Keep Monthly Change, but derive it from Demo P update metadata/change hashes.

Phase 2: Make Monthly Change a generated report, not an input to Master List update.

Phase 3: Remove or hide it only after the new Demo P/Disenrolled ledgers prove reliable.

The key design rule: Demo P and Disenrolled List must be mutually exclusive

If a PMR is in Demo P, it is active/current.

If a PMR is in Disenrolled Exclusion List, it is inactive/disenrolled.

A PMR should not appear in both, except possibly during an intermediate processing transaction.

Recommended enforcement:

During Raw Data processing:

build latest PMR state from Raw Data

if latest state = active:

remove PMR from Disenrolled Exclusion List if reinstated

upsert PMR in Demo P

if latest state = disenrolled:

remove PMR from Demo P

upsert PMR in Disenrolled Exclusion List

This matters because the current split function already treats disenrollment at the PMR-block level: once a PMR block is classified as disenrolled, it does not push the rows into the active rows array.

That is the correct behavior for the new model too.

Recommended metadata columns for ongoing Demo P

Since you want to keep the name Demo P, I would make the Demo P metadata columns the main update ledger:

Column	Required?	Purpose
Demo P Update Status	Yes	Created, Updated, Unchanged, Removed to Disenrolled, Reinstated
Demo P Update Month	Yes	Month of latest update/check
Demo P Source Sheet	Yes	Raw Data/source sheet that last touched row
Demo P Columns Updated	Yes	Replacement for much of Monthly Change
Demo P Last Updated At	Strongly recommended	Exact timestamp
Demo P Source PMR Hash	Strongly recommended	Fast change detection
Demo P Previous PMR Hash	Optional	Useful for audit/debug
Demo P Last Workflow	Optional	Format Raw Data / Process Demo P / Manual Repair

The existing code already defines the first three.

Recommended metadata columns for ongoing Disenrolled Exclusion List

For the ongoing Disenrolled List, I would add parallel columns:

Column	Purpose
Disenrolled Update Status	Newly Disenrolled, Updated, Confirmed, Reinstated/Removed
Disenrolled Update Month	Month disenrollment was first detected or most recently confirmed
Disenrolled Source Sheet	Raw Data source sheet
Disenrolled Reason Field	Enrollment Status, Disenrollment Effective Date, Date of Death
Disenrolled Effective Date	Best available disenrollment/date-of-death value
Disenrolled Last Updated At	Exact timestamp
Disenrolled Source PMR Hash	Latest source row/block hash
Demo P Removal Status	Removed from Demo P, Not Found in Demo P, Reinstated

The current code identifies disenrollment from Enrollment Status, disenrollment date, or date of death.

So these metadata fields match what the script already uses to classify a PMR.

Reinstatement handling

You should define reinstatement explicitly.

A participant could appear in the Disenrolled Exclusion List from a prior month and then appear active again in a later Raw Data import. In that case, I would not simply ignore them.

Recommended logic:

If PMR exists in Disenrolled Exclusion List
and latest Raw Data status is active
and no disenrollment/date-of-death field is present:

```

move or mark PMR as Reinstated
remove PMR from active exclusion set
upsert PMR into Demo P
set Demo P Update Status = Reinstated

```

This is safer than permanently excluding every PMR that ever appeared in the Disenrolled List.

The current split function can already accept an external disenrolled PMR set, which means the architecture has a natural place to enforce or relax historical exclusions.

For the ongoing model, I would use current Raw Data as the latest authority, but keep historical disenrollment data for audit.

Practical migration plan

Phase 1 — Rename concept, not sheets

Do not rename Demo P to Processed Data in code yet.

Instead:

Keep sheet name: Demo P

Update documentation/comments/dashboard descriptions to say:

Demo P = current active participant processed source

Disenrolled Exclusion List = current inactive/disenrolled participant source

This avoids breaking the current Master List flow, which directly depends on Demo P.

Phase 2 — Make Demo P ongoing

Change Process Demo P from “create monthly Demo P output” to “update stable Demo P.”

Key behavior:

upsert active PMRs

update only changed PMR blocks

mark unchanged PMRs

remove disenrolled PMRs

preserve existing rows when PMR unchanged

Phase 3 — Make Disenrolled Exclusion List ongoing

Change the disenrolled write path so it updates stable Disenrolled Exclusion List instead of deleting/replacing dated outputs. The current code builds a dated output name and deletes existing legacy/dated sheets before writing.

That should become:

get or create stable Disenrolled Exclusion List

load existing PMR index

upsert newly disenrolled PMRs

update metadata

do not delete prior disenrolled history

Phase 4 — Simplify Master List update

Instead of:

previous Master List
+ current Demo P
+ previous Demo P
+ Monthly Change Report
move toward:

current Master List
+ ongoing Demo P
+ ongoing Disenrolled Exclusion List
+ Demo P update metadata

The current update path still requires current Demo P, previous Demo P, and previous Master List.

Long-term, previous Demo P should disappear from the required operational path.

Phase 5 — Make Monthly Change report optional
Monthly Change can be regenerated from:

Demo P Columns Updated

Demo P Update Month

Demo P Update Status

Disenrolled Update Month

Disenrolled Update Status

So it becomes a report, not a required intermediate control sheet.

My recommended final architecture

Raw Data mm.yy

- untouched/import snapshot plus approved processing columns
- source for this cycle only

Demo P

- ongoing active participant state
- one current source for Master List
- active PMRs only
- tracks update status/month/source/columns changed/hash

Disenrolled Exclusion List

- ongoing inactive/disenrolled participant state
- one current exclusion ledger
- all known disenrolled PMRs
- tracks disenrollment month/source/reason/hash

Master List mm.yy

- operational monthly output
- generated/updated from Demo P

- excludes PMRs in Disenrolled Exclusion List
- carries Master List metadata

Monthly Change Report

- optional report
- generated from Demo P + Disenrolled metadata
- no longer required as the source of truth

Best answer to your exact proposal

"The Disenrolled List could also be ongoing. Store all disenrolled participants there and current participant in Processed Data — will keep the name Demo P for Processed Data."

I agree. I would implement that as:

Demo P = ongoing active processed data.

Disenrolled Exclusion List = ongoing disenrolled processed data.

Raw Data = monthly source import/snapshot.

Master List = generated from Demo P, excluding Disenrolled List.

Monthly Change = optional derived report, not required source.

That is cleaner than maintaining monthly Demo P sheets plus monthly change comparison sheets as the main mechanism.

Commands used to review this

```
✓ git status -sb && git log -1 --oneline --decorate && rg -n "Disenrolled|DISENROLLED|Demo
P|Processed Data|Master List Update Status|Master List Source Sheet|Primary PMR
Row|createDisenrolled|splitRawDataRowsIntoActiveAndDisenrolled|createActiveDemoPFromRawData|cr
eateMasterList|updateMasterListForMonth" src/Code.gs README.md 'spec/Process Data (1).txt' | sed -n
'1,260p'
```

```
✓ nl -ba src/Code.gs | sed -n
'245,260p;323,354p;3205,3286p;3834,3890p;3897,3928p;3930,3982p;4010,4064p;4149,4185p;5421,54
92p;5815,5838p;7297,7339p;9370,9433p' && nl -ba README.md | sed -n '82,91p;155,161p'
```

Keep Monthly Change Report for now. Remove Master List Change Log. Should we remove the secondary rows from the Processed Data or leave them? What does this mean---compute a PMR-level source hash,?? I had the Banner sync on the Raw Data because it was the only way the a change in banners would show up on the Monthly Change Log. I am leaning toward keeping the Monthly Change Log so I think it would have to stay under Format Data . Maybe make the Metadata columns universal instead of sheet specific??. There are time participants will be disenrolled and then rejoin so they would be removed from Disenrolled List. For both Demo P and Disenrolled would you need separate Create and Update triggers??

Copy

My read of your direction

I think your revised direction is coherent:

Keep Monthly Change Report for now.

Remove Master List Change Log as a separate sheet/logging dependency.

Keep the name Demo P, but treat it as the ongoing processed/current active participant source.

Make Disenrolled Exclusion List ongoing, holding PMRs that are currently disenrolled.

Support reinstatement by removing PMRs from the active Disenrolled List when they rejoin.

Keep Banner sync early enough that Banner changes still appear in Monthly Change Report.

Move toward universal metadata columns rather than sheet-specific metadata columns.

That is a good compromise: it modernizes the data model without removing Monthly Change Report before the workflow is stable.

1. Keep Monthly Change Report, remove Master List Change Log

Yes — I would keep Monthly Change Report and remove Master List Change Log.

Those two sheets serve different purposes:

Sheet	Purpose	Keep?
Monthly Change Report	Explains which PMRs changed and why; drives/reviews monthly update	Yes, for now
Master List Change Log	Low-level audit of rows deleted/changed during Master List update	Remove or replace later

The current monthly update flow already treats Monthly Change Report as the important step before Master List update: it builds Monthly Change first, then runs Master List update.

The current Master List update also reads changed PMRs from Monthly Change Report and applies actions by section: enrollments, disenrollments, demographic changes, caseload changes, banner changes, and contact changes.

By contrast, Master List Change Log is currently written during low-level deletion operations — for example, when deleting PMR blocks from the Master List, the function builds JSON snapshots of deleted rows and calls `appendMasterListChangeLog_`.

So I would remove Master List Change Log first, because its practical value can be replaced by better row-level metadata and the Monthly Change Report itself.

Important distinction

When you said, “I am leaning toward keeping the Monthly Change Log,” I think you likely mean Monthly Change Report, not Master List Change Log.

I would use these names consistently:

Monthly Change Report = keep.

Master List Change Log = remove.

Avoid "Monthly Change Log" as a name, because it creates confusion between those two.

2. Should secondary rows be removed from Demo P / Processed Data?

My recommendation: leave secondary rows in Demo P for now

I would not remove secondary rows from Demo P yet.

Reason: Demo P is not just a Master List primary-row source. It also supports contact/secondary-row logic. The current Master List update path has separate handling for primary PMR updates and contact/secondary-row merges. It updates primary rows for demographic/caseload/banner changes, then separately merges secondary/contact rows for contact changes.

That means if you remove secondary rows from Demo P too early, you risk losing the ability to correctly update contact rows in the Master List.

The current architecture also says every participant has exactly one Primary PMR Row, but it does not say every participant only has one row. Instead, synchronization targets the Primary PMR Row.

So for now:

Demo P should retain all active participant rows:

- primary row
- secondary/contact rows
- source rows needed for contact processing

But:

Master List should only copy/use the rows it needs:

- primary row for participant summary
- secondary/contact rows only where the Master List workflow requires them

The current createMasterList() flow already copies primary PMR rows from processed Demo P into the Master List.

When could secondary rows be removed?

Only remove secondary rows from Demo P if you first move all secondary-row information into a normalized primary-row structure, such as:

Contact - 1

Contact - 2

Contact - 3

Contact - Summary

Phone - Summary

Notes Summary

Some of those kinds of derived fields already exist in the workflow, but I would not assume they fully replace the secondary rows yet. The Demo P processing function still runs contact processing and notes/contact transformations against the Demo P data range.

Best staged approach

Phase Demo P secondary rows Reason

Now Keep Safest; preserves contact logic

After metadata/hash rollout Keep but hide/filter secondary rows Reduces visual clutter without data loss

Later Remove only if all contact/secondary data is normalized onto primary rows Cleaner but higher risk

So my answer is: leave secondary rows in Demo P for now.

3. What does “compute a PMR-level source hash” mean?

A PMR-level source hash is a compact fingerprint of all the source data that matters for one participant.

Instead of comparing 50–200 individual columns every month, the script can create one repeatable value like:

PMR 123456 source hash = 8f4a9c1e...

If the hash is the same as last time, the PMR did not materially change.

If the hash is different, something about that PMR changed.

Example

For one PMR, imagine Raw Data has three rows:

PMR 123456 row 1: Name, DOB, Medicaid, Address, Enrollment Status

PMR 123456 row 2: Contact A

PMR 123456 row 3: Contact B

The script would:

collect all rows for PMR 123456,

keep only fields that matter,

normalize values,

include synced Banner values if Banner changes should count,

join everything into one stable string,

compute a hash.

Conceptually:

sourceText =

"PMR=123456" +

"|Name=Smith, John" +

"|DOB=01/01/1970" +
"|Enrollment Status=Active" +
"|Address=..." +
"|Banner Summary=..." +
"|Contact1=..." +
"|Contact2=...";

sourceHash = hash(sourceText);
Then store it in a column such as:

Source Hash
or:

PMR Source Hash
Why this helps
A PMR-level hash lets you quickly decide:

Situation	Meaning
New PMR not in Demo P	Created
PMR exists and hash unchanged	Unchanged
PMR exists and hash changed	Updated
PMR now disenrolled	Move to Disenrolled List
PMR previously disenrolled but now active	Reinstated

It is not about security. It is just a fast, reliable change-detection fingerprint.

What should be included in the hash?
Include fields that should cause a Monthly Change Report entry:

demographics,

enrollment/disenrollment status,

caseload fields,

contact rows,

address/phone/language fields,

Banner values if Banner changes should show in Monthly Change Report.

Do not include volatile fields that change every run, such as:

update timestamp,

source sheet name,

row number,

formatting,

calculated metadata,

current run status.

4. Banner sync and Monthly Change Report

Your reason for syncing Banner data onto Raw Data makes sense:

Banner changes had to live somewhere before Monthly Change comparison, otherwise they would not appear as changes.

The current Raw Data formatter explicitly says it appends/populates approved Raw Data sync columns, including Primary PMR Row and Banner fields.

The current Demo P processing then builds Banner Summary from existing data and explicitly says Banner values are sourced from Raw Data, not synced into Demo P during that later process.

So your current design has this logic:

Banner Report → Raw Data sync columns → Demo P → Monthly Change Report
That is valid.

If Monthly Change Report stays, Banner sync must stay before comparison

If Monthly Change Report continues comparing previous Demo P to current Demo P, then Banner data must be present in current Demo P before Monthly Change Report is built.

Currently, Monthly Change Report compares previous Demo P to current Demo P.

So the requirement is:

Format Raw Data / Banner sync

→ Process Demo P

→ Build Monthly Change Report

Banner sync does not necessarily have to stay on Raw Data forever, but it must happen before the Monthly Change Report comparison.

Recommended near-term choice

Since you are keeping Monthly Change Report for now, I would keep Banner sync under Format Raw Data for now.

That preserves current behavior and avoids creating a blind spot where Banner changes do not show up in Monthly Change Report.

Later, once Demo P is ongoing and has source hashes, Banner sync could move to the Demo P update step — but only if the Monthly Change Report reads from Demo P metadata/hash differences instead of relying entirely on previous-vs-current sheet comparison.

5. Universal metadata columns vs sheet-specific metadata columns

I agree with moving toward universal metadata columns.

Right now the code has sheet-specific metadata columns:

Master List Update Status

Master List Update Month

Master List Source Sheet

Demo P Update Status

Demo P Update Month

Demo P Source Sheet

And there is a dedicated Master List metadata writer that only writes the Master List-specific columns.

That works, but it gets repetitive as you add ongoing Demo P and ongoing Disenrolled List.

Recommended universal metadata columns

I would standardize these across Demo P, Disenrolled Exclusion List, and Master List:

Universal column	Meaning
Update Status	Created, Updated, Unchanged, Disenrolled, Reinstated, Removed
Update Month	Reporting month for the last source update
Source Sheet	Sheet that provided the latest source data
Columns Updated	Comma-separated list of fields changed
Last Updated At	Exact timestamp
Source Hash	Current PMR-level source hash
Previous Source Hash	Prior hash before update
Source Workflow	Format Raw Data, Process Demo P, Update Master List, etc.

Why universal columns are better
They let you write one helper:

`writeMetadataColumns_(sheet, row, metadata)`
instead of separate helpers for:

`fillMasterListMetadataColumns_`
`populateDemoPUpdateColumns_`
`populateDisenrolledUpdateColumns_`
Current code already has at least two patterns: Demo P metadata population and Master List metadata population.

Universal metadata would simplify that.

But use a careful migration
I would not rip out existing sheet-specific columns immediately.

Recommended migration:

Add universal metadata columns to templates.

Continue writing old columns for compatibility.

Start writing universal columns everywhere.

Once stable, remove old sheet-specific metadata columns from templates/code.

6. Reinstatement: disenrolled participants can rejoin

Yes — this is important.

If a participant is disenrolled and later rejoins, they should not remain in the active Disenrolled Exclusion List.

I would define the Disenrolled List as:

Current disenrolled/inactive PMRs only.

So when a PMR rejoins:

Remove PMR from Disenrolled Exclusion List

Add/update PMR in Demo P

Set Update Status = Reinstated

Set Columns Updated = Enrollment Status, Disenrollment Date, etc.

But keep some history somewhere

Because you also want to remove Master List Change Log, you should decide how much history you still need.

Options:

Option Behavior	Recommendation
-----------------	----------------

Remove from Disenrolled List entirely	Clean current-state list Good operationally
---------------------------------------	--

Move to hidden Disenrolled History	Keeps reinstatement audit Best long-term
------------------------------------	---

Keep in Disenrolled List with status Reinstated	Easy audit but can confuse exclusion logic I would avoid this for active exclusion list
---	--

My recommendation:

Disenrolled Exclusion List = current exclusions only

Optional Disenrolled History = historical audit

If you do not want another sheet, at minimum Demo P should show:

Update Status = Reinstated

Source Sheet = Raw Data mm.yy

Update Month = mm.yy

Columns Updated = Enrollment Status, Disenrollment Effective Date

Previous Source Hash = ...

Source Hash = ...

7. Do Demo P and Disenrolled need separate Create and Update triggers?

User-facing answer: probably yes, but with smart combined behavior
I would expose simple menu actions like this:

Format Raw Data

Update Demo P

Update Disenrolled List

Build Monthly Change Report

Update Master List

But internally, I would not create completely separate logic for “create” and “update.” I would make each operation an upsert.

Upsert means create if missing, update if present

For Demo P:

Update Demo P:

if Demo P sheet does not exist:

create it from template

else:

update existing Demo P

For Disenrolled Exclusion List:

Update Disenrolled List:

if Disenrolled Exclusion List does not exist:

create it from template

else:

update existing list

So the user sees one action, but the code handles both cases.

Recommended menu design

Menu item Internal behavior

Create / Update Demo P Creates Demo P if missing; otherwise updates existing active ledger

Create / Update Disenrolled List Creates list if missing; otherwise upserts disenrolled PMRs

Build Monthly Change Report Uses latest Demo P state and/or previous snapshot

Update Master List Uses Monthly Change Report for now

Full Rebuild Demo P Optional advanced repair action

Full Rebuild Disenrolled List Optional advanced repair action

Why not separate create and update buttons?

Separate buttons increase user decision points:

Do I click Create?

Do I click Update?

What if it already exists?

What if I click the wrong one?

Upsert actions are safer.

But I would still keep hidden/internal functions like:

createDemoPIfMissing_()

updateDemoPFromRawData_()

```
rebuildDemoPFromRawData_()
createDisenrolledListIfMissing_()
updateDisenrolledListFromRawData_()
rebuildDisenrolledListFromRawData_()
```

That keeps the code clean while the user menu stays simple.

8. Recommended near-term architecture with your changes

Here is the revised near-term workflow I would target:

1. Format Raw Data

- preserve Raw Data
- assign Primary PMR Row
- sync Banner fields
- compute source-ready values

2. Create / Update Demo P

- active PMRs only
- keep secondary rows for now
- write universal metadata
- remove PMRs that are now disenrolled
- reinstate PMRs that rejoined

3. Create / Update Disenrolled Exclusion List

- current disenrolled PMRs only
- remove reinstated PMRs
- write universal metadata
- optional historical archive later

4. Build Monthly Change Report

- keep for now
- compare prior state to current state
- include Banner changes

5. Update Master List

- use Monthly Change Report for now
- remove Master List Change Log
- write metadata columns instead

9. Specific answers to each question

“Remove Master List Change Log?”

Yes. Remove it after ensuring Monthly Change Report and row metadata cover what you need. The current Master List Change Log is mainly written during low-level row deletions, while Monthly Change Report is already the higher-value business-facing change document.

“Should we remove secondary rows from Processed Data / Demo P or leave them?”

Leave them in Demo P for now. The Master List update process still has distinct contact/secondary-row merge logic, so deleting secondary rows risks losing contact update fidelity.

“What does PMR-level source hash mean?”

It means one deterministic fingerprint per PMR, based on all source fields that should count as a change. If the hash changes, the PMR changed. If it does not, the PMR is unchanged.

“Banner sync was on Raw Data because it was the only way Banner changes would show in Monthly Change Log.”

That reasoning is valid. Since Monthly Change Report stays, Banner sync must remain before Monthly Change comparison. Keeping Banner sync under Format Raw Data is the safest near-term choice because current Raw Data formatting already appends/populates Primary PMR Row and Banner fields.

“Maybe make metadata columns universal instead of sheet specific?”

Yes. Move toward universal metadata columns, but migrate gradually because current code already has sheet-specific Demo P and Master List metadata columns.

“Participants can be disenrolled and then rejoin, so they would be removed from Disenrolled List.”

Correct. The active Disenrolled Exclusion List should be current-state only. If a PMR rejoins, remove it from the active exclusion list and update Demo P with Update Status = Reinstated.

“For both Demo P and Disenrolled would you need separate Create and Update triggers?”

I would expose one create/update menu action for each, not separate user-facing create and update triggers:

Create / Update Demo P

Create / Update Disenrolled List

Internally, those functions should create the sheet if missing and update it if present.

Commands used to review this

```
✓ rg -n "Master List Change Log|appendMasterListChangeLog|Banner|Banner Summary|Primary PMR Row|processRawDataApprovedSyncColumns|processDemoPAsWorkingSource|Monthly Change|Columns With Change|Demo P Update|Master List Update" src/Code.gs README.md 'spec/Process Data (1).txt' | sed -n '1,260p'
```

```
✓ nl -ba src/Code.gs | sed -n '245,258p;3205,3282p;3385,3544p;3836,3868p;3883,3898p;5421,5492p;5815,5836p;7297,7339p;8010,8088p;8106,8138p;9200,9238p;9370,9433p'
```

What commit did you say was causing the large number of diff files?

Copy

The large diff was from the prior PR/branch history before we reset back to origin/main. From the terminal output you provided, the commit at the tip after that PR merged was:

616f1cb Merge pull request #64 from tabs912/Work

You showed it here:

616f1cb (HEAD -> work, origin/work, origin/main, origin/HEAD, main) Merge pull request #64 from tabs912/Work

So the best answer is:

The large number of diff files came from the work that was merged in commit 616f1cb, titled “Merge pull request #64 from tabs912/Work.”

Later, after the cleanup/reset, the current branch tip changed to:

6bb67e6 (HEAD -> work, origin/main) Merge pull request #66 from tabs912/work

That current 6bb67e6 state is the clean baseline we were reviewing from, not the original large-diff PR.

The only thing is for the Disenrolled List, I want that list to contain the contacts rollup. I have already added to the current so I would like the source of the List to be the Demo P.

Copy

Yes — if the Disenrolled List needs the contact rollup, then Demo P should be the source

I agree with that correction.

If the Disenrolled Exclusion List needs the same contact rollup fields that are already being created in Demo P, then Raw Data should not be the final source for the Disenrolled List.

The source should be:

Raw Data

- processed into Demo P-shaped data
- contacts/address/phone/language/notes/banner rollups applied
- disenrolled PMRs copied from processed Demo P data into Disenrolled Exclusion List
- disenrolled PMRs removed from active Demo P

That is better than:

Raw Data

- split disenrolled rows
- copy raw disenrolled row to Disenrolled List

because the raw split happens before Demo P contact processing/rollups are available.

Why Raw Data is not enough for the Disenrolled List

The current Raw Data split identifies disenrolled PMRs and stores only the first/primary row for each disenrolled PMR block.

That is useful for exclusion, but it is not enough if the Disenrolled List needs:

Contact - 1

Contact - 2

Contact - 3

...

Contact - 8

Contact - Summary

Those contact rollup columns exist in the current column definitions.

And Demo P processing is where contact processing is run. The Demo P working-source process builds Banner Summary, combines address fields, handles language, splits phones, runs contact processing, and combines notes summary.

So the Disenrolled List should be sourced after Demo P processing, not directly from Raw Data.

The best workflow order

I would change the conceptual processing order to this:

1. Format Raw Data
 - preserve source
 - assign Primary PMR Row
 - sync Banner fields
2. Build/update Demo P working data
 - include active + newly disenrolled rows at first
 - map Raw Data to Demo P headers
 - process contacts
 - build Contact - 1 through Contact - 8
 - build Contact - Summary
 - build Banner Summary
 - combine address / language / phone / notes fields
3. Split processed Demo P data
 - active PMRs stay in Demo P
 - disenrolled PMRs are copied to Disenrolled Exclusion List
 - reinstated PMRs are removed from Disenrolled Exclusion List and kept in Demo P
4. Build Monthly Change Report
 - compare processed Demo P states
 - keep Banner/contact/disenrollment changes visible

5. Update Master List

This preserves the idea that Demo P is the processed source.

Important implementation detail: split after rollups, not before

The current code has two relevant paths:

Path A — Raw Data split before Demo P processing

When building Demo P from Raw Data, the code splits Raw Data rows into active and disenrolled groups first.

That means disenrolled rows may be excluded before Demo P contact rollup processing happens.

Path B — Demo P split after Demo P exists

There is also a function that moves disenrolled PMRs from an existing Demo P sheet to the exclusion list. It reads Demo P rows, identifies PMRs with Enrollment Status = disenrolled, copies those PMRs to the exclusion list, and then removes them from Demo P.

That second path is closer to what you want.

For your new architecture, the correct model is:

Process into Demo P format first.

Then move/copy disenrolled PMRs from processed Demo P into Disenrolled List.

Disenrolled List should copy by matching Demo P headers

The existing exclusion writer already maps from source headers to Disenrolled Exclusion List headers by matching header names. It builds sourceIndexes from the exclusion headers and copies matching source columns into the output row.

That is good.

It means if the Disenrolled Exclusion List template contains the same rollup headers as Demo P — for example:

Contact - 1

Contact - 2

Contact - 3

...

Contact - 8

Contact - Summary

Banner Summary

Notes Summary

then copying from processed Demo P can naturally populate them.

But copying from Raw Data will not reliably populate those fields, because Raw Data does not necessarily have the same processed rollup values yet.

Recommended Disenrolled List row model

Since the list is meant to contain the contacts rollup, I would make it one row per disenrolled PMR, sourced from the processed Demo P primary row.

Recommended Disenrolled List fields:

Field group Source

PMR / name / DOB / identifiers Demo P primary row

Enrollment/disenrollment status/date Demo P primary row

Address / phone / language rollups processed Demo P primary row

Banner Summary processed Demo P primary row

Contact - 1 through Contact - 8 processed Demo P primary row

Contact - Summary processed Demo P primary row

Notes Summary processed Demo P primary row

Metadata current workflow/update logic

That gives the Disenrolled List enough context to be useful without preserving all secondary rows.

What this means for secondary rows

This actually strengthens the case for keeping secondary rows during Demo P processing.

You can still end with a one-row-per-PMR Disenrolled List, but you need the secondary/contact rows available long enough to build the contact rollup.

So the sequence should be:

Raw rows including secondary rows

- Demo P working data
- process contact rows into Contact - 1..8 / Contact Summary
- copy processed primary row to Disenrolled List
- remove disenrolled PMR block from active Demo P

Do not remove secondary rows before contact processing.

Handling participants who rejoin

Since you want reinstated participants removed from the Disenrolled List, the ongoing update should do this:

For each PMR in latest processed Demo P working data:

if active:

- upsert into Demo P
- remove from Disenrolled Exclusion List if present
- set Update Status = Reinstated if it was previously disenrolled

if disenrolled:

- upsert into Disenrolled Exclusion List from processed Demo P primary row
- remove from active Demo P

The key is that the Disenrolled List is a current exclusion list, not a permanent historical list.

If you later want history, add a separate hidden history/audit sheet. But the active Disenrolled List should only contain PMRs that should currently be excluded.

Revised architecture with your correction

Raw Data

- source snapshot
- keeps import data
- Banner sync can remain here for Monthly Change visibility

Demo P

- ongoing processed active participant source
- temporarily processes all latest PMRs before final split
- keeps active participants after split
- contains contact rollups

Disenrolled Exclusion List

- ongoing current disenrolled participant list
- source = processed Demo P primary rows
- contains contact rollups
- removes PMRs that rejoin

Monthly Change Report

- keep for now
- should compare processed Demo P states
- should include disenrollment/reinstatement/contact/banner changes

Master List

- generated/updated from Demo P
- excludes PMRs in Disenrolled Exclusion List

Practical recommendation

I would not have Create Disenrolled List read directly from Raw Data anymore.

Instead, make it either:

Create / Update Disenrolled List from Demo P
or have Create / Update Demo P do both:

Create / Update Demo P

- process Raw Data into Demo P working data
- update active Demo P
- update Disenrolled Exclusion List from processed Demo P rows

The second option is cleaner because it prevents the user from accidentally running the Disenrolled List before Demo P rollups are ready.

Best final rule

The Disenrolled List should be sourced from processed Demo P primary rows, not Raw Data rows.

That gives you the contact rollup you want and keeps Demo P as the single processed participant source.

Commands used to review this

```
✓ rg -n "Contact -  
Summary|Contact|combineNotesSummaryData|runMasterContactProcessData|processDemoPAsWorkin  
gSource|moveDisenrolledPMRsFromDemoPToExclusion|appendDisenrolledPMRBlocksToExclusion|split  
RawDataRowsIntoActiveAndDisenrolled|Demo P Working Source|Disenrolled" src/Code.gs README.md  
| sed -n '1,240p'
```

```
✓ nl -ba src/Code.gs | sed -n  
'245,258p;3385,3544p;3836,3868p;3914,3982p;4049,4147p;4149,4185p;5421,5492p;7297,7339p'
```

Is sync Banners, CP Due Date and Unlocked CP the only functions not included on the Demo P sheet
Copy

Short answer

Not exactly.

If you mean external monthly source syncs, then the main ones that are not owned by Demo P are:

Unlocked CP sync

Care Plan Due sync

Direct Banners sync from the newest Banners sheet — although Banner values do flow into Demo P indirectly through Raw Data / existing Banner fields, and Demo P builds Banner Summary.

But if you mean everything that is not calculated on Demo P, then there are additional Master List-only fields/workflows beyond those three, especially the HHA/O2/PAP/PERS-style columns shown in the Master List dashboard source map.

What Demo P currently does include

The current Demo P processing function does these things:

builds Banner Summary,

combines address fields,

processes language fields,

splits phone fields,

processes contacts,

combines notes summary,

writes the processed Demo P data back to the sheet.

The notification text also explicitly says Demo P processing includes:

contact, phone, address, language, notes, banner, and Primary PMR fields.

So Demo P is already responsible for the core participant rollup work.

Demo P-included processing

Area Included on Demo P? Evidence

Primary PMR Row Yes, assigned during Raw Data and copied into Demo P Demo P notes say Primary PMR Row is assigned during Format Raw Data and copied into Demo P.

Banner Summary Yes, built on Demo P from existing Banner fields Demo P calls updateBannerColumnData_.

Address processing Yes Demo P calls combineAddressesData_.

Language processing Yes Demo P calls handleLanguageData_.

Phone processing Yes Demo P calls splitPhoneNumbersData_.

Contact rollup Yes Demo P calls runMasterContactProcessData_.

Notes Summary Yes Demo P calls combineNotesSummaryData_.

Banners are a special case

Banners are partly included and partly not included.

What Demo P does

Demo P builds Banner Summary from Banner-related fields that are already present in its data.

The dashboard source map also shows Banner Summary as a Demo P / Process Demo P field.

What Demo P does not currently do

Demo P does not sync the newest Banners sheet directly during processDemoPAsWorkingSource_.

The code comment says:

Banner values are sourced from Raw Data; do not sync a newest Banners sheet into Demo P memory during this process.

So the current architecture is:

Banners sheet

→ Raw Data / approved Banner sync fields

→ Demo P builds Banner Summary

not:

Banners sheet

→ Demo P direct sync

That matters for your future architecture. If Demo P becomes ongoing/current-state, you can either keep Banner sync under Format Raw Data or move the direct Banner sync into the Demo P update step. But if Monthly Change Report must see Banner changes, Banner data must reach Demo P before Monthly Change Report is built.

CP Due and Unlocked CP are clearly not Demo P-owned today

The code explicitly says:

Care Plan Due and Unlocked Care Plan remain Master List sync functions.

And createMasterList() runs these after copying Demo P primary rows into Master List:

syncMasterListFromUnlockedCarePlan_

syncMasterListFromCarePlanDue_

The Master List single-pass processor also syncs monthly sources into memory, including:

Banners,

Unlocked Care Plan,

Care Plan Due.

CP / Unlocked sync details

Sync Source sheet Matching key Fields synced

Unlocked CP Unlocked CP monthly output PMR # → Participant PMR# Participant Name, PMR #, IDT Meeting Date, Care Plan Start Date

CP Due CP Due monthly output Participant Name Enrollment Date, Last Care Plan, Next Care Plan Due, CP Type

So yes: CP Due and Unlocked CP are definitely outside Demo P today.

Other things that are also not really Demo P-owned

The dashboard source mapping shows several Master List fields that do not come from Demo P or basic Demo P processing.

For example, Master List fields like:

CP Updated

Equipment

Face Sheet

HHA

HHA CP

HHA CP link

HHA ISR

HHA ISR link

O2 Auth

O2 Auth link

O2 Education

PAP Auth

PAP Auth link

PAP Education

PAP Supplies

PERS Auth

PERS Auth Link

are mapped to Sync Unlock CP, Sync Care Plan Due, extraction workflows, or blank/manual sources rather than Demo P.

So if the question is:

Are Banners, CP Due Date, and Unlocked CP the only source workflows not included on Demo P?

Then the answer is:

No. CP Due and Unlocked CP are the main monthly source syncs outside Demo P, Banners is partially upstream of Demo P, and there are additional Master List-only/extraction/manual fields that are not Demo P-owned.

Clean way to think about it

Demo P should own participant-core fields

Demo P should own:

demographics,

enrollment/disenrollment status,

PMR grouping,

Primary PMR Row,

address rollup,

phone rollup,

language rollup,

contact rollup,

notes summary,

Banner Summary if Banner changes need to show in Monthly Change Report.

That matches the current Demo P processing function.

Master List should own operational/sync-only fields

Master List should own:

CP Due fields,

Unlocked CP fields,

HHA/O2/PAP/PERS operational fields,

manual/extraction output fields,

final Master List status/update metadata.

That matches the current Master List source mapping and sync functions.

My recommendation for the revised architecture

If you are moving to an ongoing Demo P and ongoing Disenrolled List, I would define ownership like this:

Field/workflow Owner

Raw participant demographics Demo P

Enrollment / disenrollment state Demo P

Primary PMR Row Raw Data / Demo P

Banner raw flags Raw Data or Banner sync step

Banner Summary Demo P

Contact rollup Demo P

Phone/address/language/notes rollup Demo P

Disenrolled List source rows processed Demo P

CP Due Master List sync

Unlocked CP Master List sync

HHA/O2/PAP/PERS extractions Master List / downstream workflows

Monthly Change Report derived from Demo P and relevant synced fields

Master List final operational output

Bottom line

CP Due Date and Unlocked CP are definitely not included on Demo P.

Banners are partially included: Demo P builds Banner Summary, but it does not currently sync directly from the newest Banners sheet during Demo P processing. Banner values are expected to already be present upstream.

There are also other Master List-only fields besides those three, especially the HHA/O2/PAP/PERS/extraction/manual fields in the Master List source mapping.

So the cleanest statement is:

Demo P owns participant processing and rollups. Master List owns CP Due, Unlocked CP, and downstream operational/extraction fields. Banner should probably be treated as a Demo P-visible upstream sync because Monthly Change Report needs to see Banner changes.

Commands used to review this

```
✓ rg -n "Care Plan|CP Due|Unlocked|Unlock CP|Banners|Banner|sync.*Demo  
P|sync.*Master|processDemoPAsWorkingSource|Care Plan Due and Unlocked|Master List  
sync|ensure.*OutputHeaders|syncMasterListFrom|processMasterListFull|processMasterListDataOnly"  
src/Code.gs README.md | sed -n '1,280'
```

```
✓ nl -ba src/Code.gs | sed -n  
'3496,3544p;3470,3478p;7640,7656p;9070,9100p;9194,9245p;9440,9465p;10352,10363p;10527,10580  
p'
```

```
✓ nl -ba src/Code.gs | sed -n '5488,5500p;7594,7636p;7670,7774p;9871,9966p;14088,14224p'
```

Ok so leaving the CP syncs on the Master List??

Copy

Yes — I would leave the CP syncs on the Master List.

Specifically, keep these as Master List syncs, not Demo P syncs:

Care Plan Due sync

Unlocked CP sync

That matches the current architecture and is the safer design.

Why CP syncs should stay on Master List

1. The current code already treats CP syncs as Master List-only

The Demo P workflow explicitly says:

Care Plan Due and Unlocked Care Plan remain Master List sync functions.

Then createMasterList() copies rows from Demo P into Master List and runs:

syncMasterListFromUnlockedCarePlan_

syncMasterListFromCarePlanDue_

So the existing intended flow is:

Demo P

→ Master List

→ Sync Unlocked CP

→ Sync Care Plan Due

not:

Demo P

→ Sync CP fields

→ Master List

2. CP fields are operational Master List fields, not core participant-state fields

Demo P should represent the processed participant source:

demographics,

enrollment/disenrollment status,

PMR grouping,

contact rollup,

address/phone/language processing,

Banner Summary,

notes summary.

The CP fields are more operational/report-output fields:

IDT Meeting Date

Care Plan Start Date

Last Care Plan

Next Care Plan Due

CP Type

The code's sync functions show that Unlocked CP and CP Due are pulled from their own monthly outputs and matched into the Master List.

That makes more sense at the Master List layer.

3. Keeping CP syncs off Demo P keeps Demo P cleaner

If Demo P becomes the ongoing processed participant source, you want it to stay focused on:

Who is the participant?

Are they active?

What are their demographics/contact/notes/banner rollups?

Master List should answer:

What operational fields are needed for this month's working list?

What CP Due / Unlocked CP values apply?

What downstream workflow fields apply?

That separation is cleaner.

Recommended ownership

Data / workflow	Keep on Demo P?	Keep on Master List?
Demographics	Yes Copied from Demo P	
Enrollment / disenrollment status	Yes Copied from Demo P	
Primary PMR Row	Yes Copied / recalculated as needed	
Contacts rollup	Yes Copied / processed from Demo P	
Notes Summary	Yes Copied / processed from Demo P	
Banner Summary	Yes, because Monthly Change needs it	Also available on Master List
Care Plan Due	No Yes	
Unlocked CP	No Yes	
HHA/O2/PAP/PERS downstream fields	No Yes	

Final recommendation

Keep this structure:

Raw Data

→ Demo P

- participant core
- contact rollup
- banner summary
- active/disenrolled split

Demo P

→ Master List

- sync Unlocked CP
- sync Care Plan Due
- downstream operational fields

So yes: leave CP syncs on the Master List.

These are remaining Sections to rebuild do we need to adjust them

SYNC FUNCTIONS

MONTHLY CHANGE FUNCTIONS\nMASTER LIST FUNCTIONS \nDISENROLLMENT FUNCTIONS

\n

Copy

Short answer

Yes — those four rebuild sections do need adjustments, but not all at the same level.

I would rank them like this:

Section	Needs adjustment?	Size of adjustment	Why
SYNC FUNCTIONS	Yes	Medium	Keep CP syncs on Master List; decide exactly where Banner sync lives.
MONTHLY CHANGE FUNCTIONS	Yes	Medium/Large	Keep report, but adapt it to ongoing Demo P / metadata / snapshots.
MASTER LIST FUNCTIONS	Yes	Medium	Source from ongoing Demo P and current Disenrolled List; remove Master List Change Log.
DISENROLLMENT FUNCTIONS	Yes	Large	Change source to processed Demo P, make list ongoing, support reinstatement.

The biggest adjustment is Disenrollment Functions because we changed the source of the Disenrolled List from Raw Data to processed Demo P, and because the list becomes ongoing/current-state.

1. SYNC FUNCTIONS — adjust, but keep CP syncs on Master List

What should stay the same

Keep these as Master List syncs:

Unlocked CP

Care Plan Due

The current code already has syncMasterListMonthlySourcesIntoData__() calling:

syncBannerSourceIntoData__

syncUnlockedCarePlanSourceIntoData__

syncCarePlanDueSourceIntoData__

And the Demo P notification explicitly says Care Plan Due and Unlocked Care Plan remain Master List sync functions.

So for the rebuild:

CP Due sync: leave on Master List
Unlocked CP sync: leave on Master List
What needs adjustment
The main decision is Banner sync.

Right now, Raw Data appends/populates approved sync columns including Primary PMR Row and Banner fields.

Demo P then builds Banner Summary, but it does not directly sync the newest Banners sheet during Demo P processing. The code comment says Banner values are sourced from Raw Data and should not be synced from newest Banners directly into Demo P memory at that point.

Recommended adjustment
Keep this logic for now:

Banner source sync
→ Format Raw Data
→ Demo P builds Banner Summary
→ Monthly Change can see Banner changes
Do not move CP Due or Unlocked CP into Demo P.

Sync Functions rebuild checklist
Keep syncUnlockedCarePlanSourceIntoData_() as Master List-only.

Keep syncCarePlanDueSourceIntoData_() as Master List-only.

Keep Banner visible before Monthly Change Report.

If Demo P becomes ongoing, ensure Banner changes update Demo P metadata/hash.

Do not let CP Due / Unlocked CP affect Demo P source hashes unless you intentionally want CP changes to count as Demo P changes.

2. MONTHLY CHANGE FUNCTIONS — keep, but adjust source model

Current behavior

Current Monthly Change functions compare previous Demo P to current Demo P. The code locates current and previous Demo P sheets, then calls compareRawDemoPForSectionReport_().

The update path also builds or fetches a Monthly Change Report before rebuilding changed PMRs on the Master List.

So the current model is:

Previous monthly Demo P

vs

Current monthly Demo P

→ Monthly Change Report

→ Master List update

What changes with ongoing Demo P

If Demo P becomes ongoing, there may no longer be a natural “previous month Demo P sheet” to compare against.

You have two choices:

Option A — keep monthly Demo P snapshots for Monthly Change only
This is the least disruptive.

Ongoing Demo P = current active participant source
Monthly Demo P Snapshot = frozen copy for comparison/reporting
Monthly Change Report can continue comparing:

Previous Demo P snapshot
vs
Current Demo P snapshot
This keeps the report logic closer to what exists today.

Pros
Lower code risk.

Existing Monthly Change comparison logic can mostly survive.

Easy for users to audit month-over-month differences.

Cons
Still creates monthly snapshot sheets.

Not as clean as a fully metadata-driven model.

Option B — make Monthly Change metadata/hash-driven
This is cleaner long-term.

Demo P row metadata:

Update Status
Update Month
Columns Updated
Previous Source Hash
Source Hash

Monthly Change Report becomes:

Read Demo P rows where Update Month = selected month
Group rows by Update Status / Columns Updated
Generate report sections

Pros
No need for monthly Demo P sheet comparisons.

Works naturally with ongoing Demo P.

Better performance.

Cons

Bigger rebuild.

Requires reliable metadata and hash logic first.

My recommendation

For now:

Keep Monthly Change Report, but use Option A initially: maintain monthly snapshots or snapshot-like comparison data while the ongoing Demo P architecture stabilizes.

Then later move to Option B.

Monthly Change Functions rebuild checklist

Keep the Monthly Change Report.

Keep sections:

Enrollments

Disenrollments

Demographic Changes

Caseload Changes

Banner Changes

Contact Changes

Ensure Banner changes still appear.

Decide whether current comparison source is:

previous/current monthly Demo P snapshots, or

ongoing Demo P metadata/hash.

Eventually remove dependence on previous monthly Demo P once metadata is proven reliable.

3. MASTER LIST FUNCTIONS — adjust source and remove Master List Change Log

Current behavior

Current Master List creation locates the current processed Demo P sheet, creates a Master List from template, copies primary PMR rows from Demo P, then fills Master List metadata.

Current monthly update copies previous Master List to current month, gets/builds Monthly Change Report, then rebuilds changed PMRs.

Master List update then applies actions from Monthly Change sections: enrollment, disenrollment, primary updates, and contact changes.

What needs adjustment

A. Source should become ongoing Demo P

Master List should treat Demo P as the current active participant source, not necessarily a newly created monthly Demo P.

Master List source = ongoing Demo P active PMRs

B. Disenrolled List should be an exclusion/current-state source

Master List should also check:

Disenrolled Exclusion List

so active Master List output does not retain currently disenrolled PMRs.

C. Remove Master List Change Log

You said remove Master List Change Log. That means Master List functions should no longer write low-level row deletion JSON logs. Instead, use:

Monthly Change Report,

universal metadata columns,

optional Dashboard Quality/Framework Timing diagnostics.

D. Keep CP syncs on Master List

Master List should continue to sync:

Unlocked CP,

Care Plan Due.

Master List Functions rebuild checklist

Use ongoing Demo P as active source.

Use Disenrolled List as exclusion/current inactive source.

Keep CP Due sync on Master List.

Keep Unlocked CP sync on Master List.

Remove Master List Change Log writer/use.

Preserve Monthly Change Report as the business-facing change report.

Fill universal metadata columns on Master List rows.

Keep Primary PMR Row handling aligned with the architecture that every participant has exactly one primary row.

4. DISENROLLMENT FUNCTIONS — biggest adjustment

This section needs the most change.

Current behavior

There are two different patterns currently visible:

Raw Data based split

The code can split Raw Data into active/disenrolled rows and stores the first row of each disenrolled PMR block.

Demo P based move

There is also a function that reads Demo P, finds PMRs where Enrollment Status is disenrolled, copies those PMR rows to the exclusion list, and removes them from Demo P.

The Demo P-based path is closer to what you want now.

New desired behavior

Because you want the Disenrolled List to contain the contact rollup, the Disenrolled List must be sourced from processed Demo P, not raw rows.

Demo P is where contact rollups are produced by `runMasterContactProcessData_()`, and where notes/contact/phone/address processing happens.

So the new order should be:

Raw Data

- Demo P working data
- process contacts / phones / notes / banner / address
- copy disenrolled primary rows from processed Demo P to Disenrolled List
- remove disenrolled PMRs from active Demo P

Ongoing Disenrolled List behavior

The Disenrolled List should become current-state:

If PMR is currently disenrolled:

 upsert into Disenrolled List from processed Demo P primary row

If PMR rejoins:

- remove from Disenrolled List
- upsert/update active Demo P

Do not make a new monthly Disenrolled List unless you want a snapshot/report copy.

Disenrollment Functions rebuild checklist

Source Disenrolled List from processed Demo P, not Raw Data.

Ensure contact rollup columns are present on the Disenrolled List template.

Make Disenrolled List ongoing/current-state.

Upsert PMRs instead of delete/recreate the whole list.

Remove reinstated PMRs from Disenrolled List.

Decide whether to maintain optional Disenrolled History separately.

Make Create / Update Disenrolled List an upsert workflow.

Prefer calling the Disenrolled update as part of Demo P update, so it always runs after rollups are ready.

Recommended rebuild order

I would rebuild in this order:

Step 1 — Disenrollment Functions

Because this changes the data model most.

Target behavior:

Processed Demo P → Disenrolled List

not:

Raw Data → Disenrolled List

Step 2 — Sync Functions

Clarify ownership:

Banner → before/inside Demo P enough for Monthly Change

CP Due → Master List

Unlocked CP → Master List

Step 3 — Master List Functions

Update Master List to use:

Demo P = active source

Disenrolled List = exclusion source

Monthly Change Report = change action/report source for now

Remove Master List Change Log.

Step 4 — Monthly Change Functions

Keep report, but decide whether it is:

snapshot comparison

or:

metadata/hash-driven

I recommend snapshot comparison first, metadata/hash-driven later.

Section-by-section final recommendation

Section	Adjust?	Final target
---------	---------	--------------

SYNC FUNCTIONS	Yes	CP Due and Unlocked CP stay on Master List. Banner remains visible before Monthly Change and Demo P summary.
----------------	-----	--

MONTHLY CHANGE FUNCTIONS Yes Keep report; initially compare snapshots, later generate from metadata/hash.

MASTER LIST FUNCTIONS Yes Source from ongoing Demo P; exclude current Disenrolled List; remove Master List Change Log; keep CP syncs.

DISENROLLMENT FUNCTIONS Yes Source from processed Demo P; make ongoing; include contact rollup; support reinstatement/removal.

Commands used to review this

✓ `git status -sb && rg -n`

```
"syncMasterListMonthlySourcesIntoData|syncBannerSourceIntoData|syncUnlockedCarePlanSourceIntoData|syncCarePlanDueSourceIntoData|buildMonthlyChangeReportFromDemoP|getOrBuildMonthlyChangeReport_|compareRawDemoPForSectionReport_|createMasterList|updateMasterListForMonth_|rebuildChangedPMRsOnExistingMaster_|updateMasterListFromMonthlyChangeActions_|createDisenrolledList|moveDisenrolledPMRsFromDemoPToExclusion_|appendDisenrolledPMRBlocksToExclusion_|processDemoPAsWorkingSource|Primary PMR Row|Care Plan Due and Unlocked" src/Code.gs README.md | sed -n '1,260p'
```

Option B — make Monthly Change metadata/hash-driven
Do we want to add and Demo P Functions?
Copy

Yes — if we choose Option B, we should add a dedicated Demo P Functions rebuild section

If Monthly Change becomes metadata/hash-driven, then Demo P is no longer just an intermediate output sheet. It becomes the current active participant state ledger and the source of the Monthly Change metadata.

So yes, I would add:

DEMO P FUNCTIONS

as its own rebuild section.

In fact, with the architecture we have been outlining, Demo P Functions should probably be rebuilt before Monthly Change Functions, because Monthly Change would depend on Demo P metadata/hash values.

Why Demo P Functions needs its own section

Right now, Demo P already does a lot:

builds Banner Summary,

combines address fields,

processes language,

splits phone fields,

processes contacts,

combines Notes Summary,

writes processed Demo P data.

The dashboard source mapping also shows these fields as Process Demo P outputs:

Banner Summary

phone fields,

address fields,

notes,

Contact - 1 through Contact - 8,

Contact - Summary,

Demo P Update Status,

Demo P Update Month,

Demo P Source Sheet.

So Demo P is already a processing hub. If we make Monthly Change metadata/hash-driven, Demo P becomes even more central.

What Demo P Functions should own

I would define the new Demo P Functions section as responsible for:

1. Ongoing Demo P state

Instead of treating Demo P as only a monthly output, Demo P Functions should manage the ongoing active participant source.

Current code still builds monthly Demo P names and renames Demo P by month.

The rebuild should decide whether Demo P is:

Demo P

or still:

Demo P mm.yy - mm.yy snapshot

For the new architecture, I would make:

Demo P = ongoing active participant source
and optionally create snapshots separately if needed.

2. Upsert active PMRs

Demo P Functions should handle:

if PMR is new active:

add to Demo P

if PMR exists and changed:

update Demo P

if PMR exists and unchanged:
keep as-is / mark unchanged

if PMR is disenrolled:
move/copy processed row to Disenrolled List
remove from active Demo P

The current code creates active Demo P from Raw Data by mapping active Raw Data rows into Demo P headers and then processing Demo P.

The rebuild should turn that into an ongoing update/upsert flow.

3. Demo P metadata columns
Demo P already has these metadata columns:

Demo P Update Status

Demo P Update Month

Demo P Source Sheet

For Option B, Demo P Functions should expand that into universal metadata, such as:

Column	Purpose
Update Status	Created, Updated, Unchanged, Disenrolled, Reinstated
Update Month	Month of latest update
Source Sheet	Raw Data / source sheet that last touched the row
Columns Updated	Fields changed this cycle
Source Hash	Current PMR-level source hash
Previous Source Hash	Previous hash before update
Last Updated At	Exact timestamp
Source Workflow	Format Raw Data / Update Demo P / Manual Repair

This is the foundation for metadata/hash-driven Monthly Change.

4. PMR-level source hash
Demo P Functions should compute and store source hashes because Monthly Change would use those hashes.

Recommended fields:

Source Hash
Previous Source Hash
Columns Updated
Update Status
Update Month

Then Monthly Change no longer needs to compare full previous/current Demo P sheets forever. It can read:

Demo P rows where Update Month = selected month
and group by:

Update Status
Columns Updated

5. Columns With Change

Monthly Change already has a Columns With Change concept. The current code ensures the Monthly Change headers include Columns With Change, and the dashboard mapping shows it as a process-populated Monthly Change field.

With Option B, Demo P should probably own the raw change list first:

Demo P.Columns Updated
Then Monthly Change can copy or summarize that into:

Monthly Change Report.Columns With Change
That gives one source of truth for changed fields.

Revised rebuild section list
I would update the rebuild plan from this:

SYNC FUNCTIONS
MONTHLY CHANGE FUNCTIONS
MASTER LIST FUNCTIONS
DISENROLLMENT FUNCTIONS
to this:

DEMO P FUNCTIONS
SYNC FUNCTIONS
DISENROLLMENT FUNCTIONS
MONTHLY CHANGE FUNCTIONS
MASTER LIST FUNCTIONS
Or, if ordered by dependency:

1. DEMO P FUNCTIONS
2. SYNC FUNCTIONS
3. DISENROLLMENT FUNCTIONS
4. MONTHLY CHANGE FUNCTIONS
5. MASTER LIST FUNCTIONS

Recommended dependency order

1. Demo P Functions
- Demo P becomes the processed active participant ledger.

It should:

process Raw Data into Demo P-shaped records,

build contact rollups,

build Banner Summary,

compute metadata/hash,

detect changes,

identify active/disenrolled/reinstated PMRs.

This must happen before metadata-driven Monthly Change.

2. Sync Functions

Sync ownership should be clarified:

Sync Owner

BannerMust feed Demo P before Monthly Change

CP Due Master List

Unlocked CP Master List

CP Due and Unlocked CP should remain Master List sync functions. The current code explicitly says that, and the Master List sync functions already call them.

3. Disenrollment Functions

Disenrollment should use processed Demo P as source, because the Disenrolled List needs contact rollup.

So Disenrollment Functions depend on Demo P Functions.

Processed Demo P row

→ Disenrolled Exclusion List

not:

Raw Data row

→ Disenrolled Exclusion List

The current Raw Data based create path still reads Raw Data directly and writes primary rows only.

That should be adjusted.

4. Monthly Change Functions

If Option B is selected, Monthly Change should read from Demo P metadata/hash:

Demo P Update Month

Update Status

Columns Updated

Source Hash

Previous Source Hash

Current Monthly Change still expects previous/current Demo P sheets.

So this section would need meaningful adjustment.

5. Master List Functions

Master List should continue to:

use Demo P as active source,

use Disenrolled List as exclusion source,

use Monthly Change Report for now,

keep CP Due and Unlocked CP syncs,

remove Master List Change Log.

The current update flow still gets/builds Monthly Change Report and rebuilds changed PMRs from it.

That can remain initially, but its source should shift from previous/current Demo P comparison to Demo P metadata/hash-derived report rows.

What the new Demo P Functions section should include
Here is the section outline I would use:

DEMO P FUNCTIONS

1. Create / Update Demo P
 - create if missing
 - update if existing
 - preserve active participant state
2. Build Demo P Working Data
 - map Raw Data to Demo P headers
 - include active + newly disenrolled rows until split step
3. Process Demo P Rollups
 - Banner Summary
 - phone fields
 - address fields
 - language fields
 - contacts
 - Contact Summary
 - Notes Summary
4. Compute PMR Source Hash
 - current hash
 - previous hash
 - changed columns
5. Write Demo P Metadata
 - Update Status
 - Update Month
 - Source Sheet

- Columns Updated
- Source Hash
- Previous Source Hash
- Last Updated At

6. Classify PMR State

- active
- disenrolled
- reinstated
- unchanged

7. Provide Outputs To Other Sections

- active rows to Master List
- disenrolled rows to Disenrolled List
- change metadata to Monthly Change Report

Final recommendation

Yes — add DEMO P FUNCTIONS.

With Option B, the revised rebuild list should be:

DEMO P FUNCTIONS

SYNC FUNCTIONS

DISENROLLMENT FUNCTIONS

MONTHLY CHANGE FUNCTIONS

MASTER LIST FUNCTIONS

Demo P Functions become the foundation because they produce:

processed participant rows,

contact rollups,

Banner Summary,

active/disenrolled/reinstated classification,

metadata,

source hashes,

changed column lists.

Monthly Change Functions should then consume Demo P metadata instead of doing all comparison work independently.

Commands used to review this

 rg -n

"processDemoP\\(|processDemoPAsWorkingSource_|populateDemoPUpdateColumns_|createActiveDe
moPFromRawData_|getOrCreateDemoPProcessingSheet_|buildMonthlyChangeReportFromDemoP|com


```
pareRawDemoPForSectionReport_|getOrBuildMonthlyChangeReport_|Primary PMR Row|Demo P  
Update|Columns With Change|Source Hash|Hash" src/Code.gs README.md | sed -n '1,260p'
```

```
✓ nl -ba src/Code.gs | sed -n  
'3385,3544p;3616,3868p;3883,3898p;6227,6273p;7297,7339p;14088,14116p;14150,14157p'
```

```
cat .codex/config.json\ncat .codexconfig\ncat .codex/settings.json\ngh codespace codex logs  
Copy
```

I ran the requested commands. None of the Codex config files were present in this repo/workspace, and the GitHub CLI command could not run because gh is not installed in the environment.

Command results

✗ cat .codex/config.json — file not found.

✗ cat .codexconfig — file not found.

✗ cat .codex/settings.json — file not found.

⚠ gh codespace codex logs — environment limitation: gh command is not installed.

I also had one initial shell formatting mistake with printf because the format string began with ---; I reran the commands with a safe printf '%s\n' ... wrapper and got the results above.

Build next section

Copy

Building the next section: DEMO P FUNCTIONS

I'm treating Demo P Functions as the next section because we just decided that if Monthly Change becomes metadata/hash-driven, Demo P has to become the source of the metadata and changed-column information.

This section should be rebuilt before Monthly Change, Master List, and Disenrollment because it becomes the central active-participant state layer.

DEMO P FUNCTIONS

Purpose

The Demo P Functions section owns the ongoing active participant source.

Demo P should no longer be treated only as a temporary monthly processed output. It should become the current active participant ledger that:

receives latest Raw Data,

processes participant rollups,

tracks row/PMR update metadata,

computes PMR-level source hashes,

identifies active, disenrolled, reinstated, created, updated, and unchanged PMRs,

supplies processed rows to Monthly Change, Master List, and Disenrolled List workflows.

The current Demo P processing already performs the core participant rollups: Banner Summary, address, language, phones, contacts, and Notes Summary.

Current behavior to replace or refactor

Current processDemoP()

Current processDemoP() does the following:

prompts for a reporting month,

gets or creates a Demo P processing sheet,

moves disenrolled PMRs to exclusion if the source was already Demo P,

processes Demo P as a working source,

applies template formatting,

renames the processed sheet to a monthly Demo P sheet.

That still behaves like a monthly-output process.

The new Demo P architecture should instead behave like:

Create / Update ongoing Demo P

not:

Create monthly Demo P mm.yy - mm.yy

Target behavior

New main workflow

Recommended user-facing function:

Create / Update Demo P

Recommended internal behavior:

1. Load latest formatted Raw Data.
2. Load existing ongoing Demo P if present.
3. Build Demo P-shaped working data from Raw Data.
4. Run Demo P processing/rollups in memory.
5. Compute PMR-level hashes and changed columns.
6. Classify each PMR:
 - Created
 - Updated
 - Unchanged
 - Disenrolled
 - Reinstated

7. Upsert active PMRs into Demo P.
8. Send processed disenrolled PMRs to Disenrolled List.
9. Remove reinstated PMRs from Disenrolled List.
10. Write metadata columns.

Demo P responsibilities

1. Build Demo P working data from Raw Data

Current code already maps Raw Data rows into Demo P headers inside `createActiveDemoPFromRawData_()`.

However, the current flow splits out disenrolled rows before Demo P processing. That is no longer ideal because the Disenrolled List needs processed Demo P contact rollups.

New rule

Demo P working data should initially include:

active PMRs

+

newly disenrolled PMRs

+

possibly reinstated PMRs

Then process all rollups first.

After processing, split:

active / reinstated → Demo P

disenrolled → Disenrolled List

2. Process participant rollups

Demo P should continue to own:

Banner Summary,

address processing,

language processing,

phone processing,

contact processing,

Notes Summary.

The dashboard source map already treats these fields as Process Demo P outputs.

Demo P-owned rollup fields

Field / group	Owner
---------------	-------

Banner Summary	Demo P
----------------	--------

phone split fields	Demo P
--------------------	--------

address rollup fields	Demo P
-----------------------	--------

notes fields	Demo P
--------------	--------

Contact - 1 through Contact - 8 Demo P
Contact - Summary Demo P
Primary PMR Row Raw Data assignment, copied/validated in Demo P
3. Compute PMR source hash
Demo P Functions should introduce a PMR-level source hash.

Purpose

The source hash tells us whether a PMR changed since the last Demo P update.

same hash = unchanged

different hash = changed

missing previous hash = created

previously disenrolled + active now = reinstated

active before + disenrolled now = disenrolled

What should go into the hash

Include fields that should trigger Monthly Change:

demographic fields,

enrollment/disenrollment fields,

caseload fields,

Banner fields / Banner Summary,

contact rows,

contact rollup fields,

address fields,

phone fields,

language fields,

notes fields.

Exclude volatile fields:

Update Status,

Update Month,

Source Sheet,

Last Updated At,

row number,

formatting,

runtime diagnostics.

Recommended helper names

buildDemoPPMRSourceHash_(pmrRows, headers, headerMap)

buildDemoPComparableObjectForPMR_(pmrRows, headers, headerMap)

normalizeDemoPHashValue_(value)

4. Track changed columns

Demo P should calculate changed fields and write them into a metadata column.

Recommended column:

Columns Updated

or if keeping sheet-specific naming initially:

Demo P Columns Updated

Monthly Change already has the concept of Columns With Change. The current code ensures Monthly Change headers include Columns With Change, and the dashboard source map shows it as process-populated.

Recommendation

Use:

Demo P.Columns Updated

as the row-level source, then copy/summarize it into:

Monthly Change Report.Columns With Change

5. Write metadata columns

Current Demo P already has:

Demo P Update Status

Demo P Update Month

Demo P Source Sheet.

For the rebuild, I recommend moving to universal metadata columns.

Recommended universal metadata

Column	Purpose
--------	---------

Update Status	Created, Updated, Unchanged, Disenrolled, Reinstated
---------------	--

Update Month	Reporting month for latest update
--------------	-----------------------------------

Source Sheet	Raw Data/source sheet used
--------------	----------------------------

Columns Updated	Changed fields
-----------------	----------------

Source Hash	Current PMR hash
-------------	------------------

Previous Source Hash	Previous PMR hash
----------------------	-------------------

Last Updated At	Timestamp
-----------------	-----------

Source Workflow	Create / Update Demo P
-----------------	------------------------

Transitional compatibility

For now, write both:

Demo P Update Status

Demo P Update Month

Demo P Source Sheet

and:

Update Status

Update Month

Source Sheet

Columns Updated

Source Hash

Previous Source Hash

Last Updated At

Source Workflow

Then later remove sheet-specific columns after the workflow is stable.

PMR state classification

Demo P Functions should classify every PMR into one of these states.

Status	Meaning	Result
--------	---------	--------

Created	PMR not previously in Demo P or Disenrolled List	Add to Demo P
---------	--	---------------

Updated	PMR exists and hash/columns changed	Update Demo P
---------	-------------------------------------	---------------

Unchanged	PMR exists and hash unchanged	Keep existing Demo P row/block
-----------	-------------------------------	--------------------------------

Disenrolled	PMR active before, latest source says disenrolled	Move to Disenrolled List and remove from active Demo P
-------------	---	--

Reinstated	PMR was in Disenrolled List, latest source says active	Remove from Disenrolled List and add/update Demo P
------------	--	--

Removed	PMR missing from latest source and no active/disenrolled state found	Optional; probably manual review
---------	--	----------------------------------

Ongoing Demo P update model

Read inputs

Latest Raw Data

Existing Demo P

Existing Disenrolled Exclusion List

Banners fields already present via Raw Data / Banner sync

Build maps

existingDemoByPMR

existingDisenrolledByPMR

incomingRawByPMR

Process incoming data

for each incoming PMR:

map Raw Data → Demo P headers

run Demo P rollups

compute source hash

compare to existing hash

classify PMR status

Write outputs

Demo P:

active and reinstated PMRs only

Disenrolled Exclusion List:

currently disenrolled PMRs only

Monthly Change:

generated later from Demo P metadata

Disenrollment handoff

Demo P Functions should produce a clear handoff object for Disenrollment Functions.

Example:

```
{  
  activeRowsByPMR: Map,  
  disenrolledRowsByPMR: Map,  
  reinstatedPMRs: Set,  
  updatedPMRs: Set,  
  createdPMRs: Set,  
  unchangedPMRs: Set  
}
```

Disenrollment Functions should then upsert rows from:

disenrolledRowsByPMR
into the Disenrolled List.

This matters because the Disenrolled List needs the processed contact rollup, which Demo P produces. Demo P processing currently runs contact processing and notes/contact rollups.

Monthly Change handoff

Demo P Functions should also produce Monthly Change-ready metadata:

PMR

Update Status

Update Month

Columns Updated

Source Sheet

Source Hash

Previous Source Hash

Then Monthly Change Functions can build sections from that metadata instead of doing full sheet comparisons.

Current Monthly Change still compares previous Demo P to current Demo P.

This section rebuild would prepare for replacing that comparison.

Recommended function map

Public / menu functions

function createOrUpdateDemoP()

```

function rebuildDemoPFromRawData()
function validateDemoPState()
Internal functions
function getOrCreateOngoingDemoPSheet_()
function buildDemoPWorkingDataFromRawData_(rawSheet, monthParts)
function processDemoPWorkingData_(workingData, monthParts)
function buildDemoPExistingStateMap_(demoSheet)
function classifyDemoPPMRUpdates_(incomingByPMR, existingByPMR, disenrolledByPMR,
monthParts)
function computeDemoPPMRSourceHash_(pmrRows, headers, headerMap)
function getDemoPChangedColumns_(previousComparable, currentComparable)
function writeDemoPMetadata_(row, metadata, headerMap)
function upsertDemoPActiveRows_(demoSheet, classifiedState)
function buildDemoPDisenrollmentPayload_(classifiedState)
function buildDemoPMonthlyChangePayload_(classifiedState)
Recommended output object
The Demo P update should return a result object, not just a sheet.

```

```

{
  demoSheet: sheet,
  sourceSheetName: rawSheet.getName(),
  monthParts: monthParts,

  createdPMRs: Set,
  updatedPMRs: Set,
  unchangedPMRs: Set,
  disenrolledPMRs: Set,
  reinstatedPMRs: Set,

  activeRowsByPMR: Map,
  disenrolledRowsByPMR: Map,

  monthlyChangeRows: [],
  metadata: {
    rowsRead: number,
    rowsWritten: number,
    hashesCompared: number,
    columnsChanged: number
  }
}

```

This lets the next sections consume Demo P results cleanly.

Dashboard/template changes needed
 Add universal metadata columns to Demo P
 Add to Demo P template:

Update Status
 Update Month
 Source Sheet

Columns Updated
Source Hash
Previous Source Hash
Last Updated At
Source Workflow
Keep old columns temporarily:

Demo P Update Status
Demo P Update Month
Demo P Source Sheet
Keep existing rollup columns
Do not remove:

Contact - 1
Contact - 2
...
Contact - 8
Contact - Summary
Banner Summary
Notes
Primary PMR Row
Those are required by Disenrolled List and Monthly Change.

Menu label recommendation
Replace or supplement:

Process Demo P
with:

Create / Update Demo P
Keep an advanced action:

Rebuild Demo P From Raw Data
The normal user should not have to decide whether to create or update. The function should create if missing and update if present.

Interaction with other rebuild sections
Sync Functions
Demo P depends on Banner being present early enough to produce Banner Summary.

CP Due and Unlocked CP stay with Master List, not Demo P. Current Demo P messaging already says Care Plan Due and Unlocked Care Plan remain Master List sync functions.

Disenrollment Functions
Disenrollment consumes processed Demo P disenrolled rows.

Monthly Change Functions
Monthly Change consumes Demo P metadata/hash and changed-column lists.

Master List Functions

Master List consumes active Demo P rows and Disenrolled List exclusions.

Acceptance criteria

The rebuilt Demo P section is acceptable when:

Demo P can be created if missing.

Demo P can be updated if it already exists.

Demo P remains the active participant source.

Disenrolled PMRs are not left in active Demo P.

Reinstated PMRs are removed from Disenrolled List and added back to Demo P.

Contact rollups are present before Disenrolled List receives rows.

Banner Summary is present before Monthly Change is generated.

Metadata columns are populated.

Source hashes are stable across repeated runs with unchanged data.

Monthly Change can be generated from Demo P metadata.

Final next-section output

DEMO P FUNCTIONS

Purpose:

Maintain the ongoing active participant source and produce metadata/hash outputs for Monthly Change, Disenrollment, and Master List workflows.

Main user workflow:

Create / Update Demo P

Core responsibilities:

- Build Demo P working data from Raw Data
- Process Banner Summary, address, phone, language, contacts, notes
- Compute PMR-level source hashes
- Identify changed columns
- Write metadata columns
- Classify Created / Updated / Unchanged / Disenrolled / Reinstated PMRs
- Upsert active PMRs into Demo P
- Provide processed disenrolled rows to Disenrollment Functions
- Provide metadata rows to Monthly Change Functions

Dependencies:

- Raw Data formatted

- Primary PMR Row available
- Banner fields available before Demo P update if Banner changes must hit Monthly Change

Outputs:

- Ongoing Demo P active participant sheet
- Demo P metadata/hash state
- Disenrollment payload
- Monthly Change payload

Commands used to review this

✓ `git status -sb && nl -ba src/Code.gs | sed -n '3385,3544p;3616,3868p;3883,3898p;6227,6273p;7297,7339p;14088,14116p;14150,14157p'`

ok

Copy

Got it.

The next rebuild sequence we settled on is:

Demo P Functions

Make Demo P the ongoing active participant source.

Add/update metadata and PMR-level hash logic.

Produce changed-column data for Monthly Change.

Provide processed disenrolled rows to the Disenrolled List.

Sync Functions

Keep CP Due and Unlocked CP on Master List.

Keep Banner available before Demo P / Monthly Change.

Disenrollment Functions

Source Disenrolled List from processed Demo P.

Make Disenrolled List ongoing.

Support reinstatement/removal from the list.

Monthly Change Functions

Keep Monthly Change Report.

Move toward metadata/hash-driven sections from Demo P.

Master List Functions

Source active participants from Demo P.

Exclude current Disenrolled List.

Keep CP syncs on Master List.

Remove Master List Change Log.

No code changes were made in this turn.

ok

Copy

Sounds good. I'll keep the agreed direction as the working plan and won't make code changes until you explicitly ask me to implement the next section.

Current agreed rebuild direction:

Keep Demo P as the ongoing active participant processed sheet.

Source the Disenrolled List from processed Demo P, so it includes the contact rollup.

Keep Monthly Change Report for now.

Remove/stop using the Master List Change Log.

Keep CP Due Date and Unlocked CP syncs on the Master List.

Keep banner updates upstream enough to feed Monthly Change visibility.

Add universal metadata/hash columns so Monthly Change can eventually be metadata/hash-driven.

Build remaining sections in this order:

Demo P Functions

Sync Functions

Disenrollment Functions

Monthly Change Functions

Master List Functions

Build Demo P

Copy

Summary

Built Demo P as an ongoing sheet by changing the Demo P processing target to the stable Demo P name and preserving that name after processing instead of renaming it to a monthly Demo P output.

Added universal metadata columns for Demo P tracking: Update Status, Update Month, Source Sheet, Columns Updated, Last Updated At, Source Hash, Previous Source Hash, and Source Workflow.

Wired Demo P processing to populate both the legacy Demo P update fields and the new universal metadata/hash fields during create/update processing.

Added PMR-level source hash helpers that exclude volatile metadata fields, normalize date/text values, and generate stable hashes for future metadata/hash-driven Monthly Change logic.

Updated the default dashboard/header mapping so rebuilt Demo P templates include the new universal metadata fields.

Testing

✓ node --check /tmp/Code.js

⚠ git diff --check — warning due existing CRLF/mixed-line-ending behavior in src/Code.gs; syntax validation passed.

✓ git status -sb

Committed changes on the current branch with commit 41bf426 and created the pull request record: Build ongoing Demo P metadata.

File (1)

Did you create the Script and Current Production files

Copy

No — I did not create or update any new scripts/ production script file or any Current_Production/ file in my latest “Build Demo P” change.

What I actually committed was only:

src/Code.gs

The latest commit confirms that commit 41bf426 Build ongoing Demo P metadata contains only src/Code.gs. I verified that with git show --stat --oneline --name-only HEAD.

The scripts/ and Current_Production/ files that are currently present in the repo already existed in the branch history before my latest Demo P commit. I listed them with find Current_Production scripts -maxdepth 1 -type f | sort, and they include files like:

Current_Production/v1.5.53_Production_Script.txt

Current_Production/v1.5.54_Production_Script.txt

Current_Production/v1.5.55_Current_Production_Script

scripts/v1.5.53_Production_Script.txt

scripts/v1.5.54_Production_Script.txt

the scripts/v1.5.55_Rebuild_*.txt files

But those were not created by this latest Demo P commit. The only committed file from my latest work was src/Code.gs.

Commands used

✓ git show --stat --oneline --name-only HEAD

✓ git status -sb

✓ find Current_Production scripts -maxdepth 1 -type f | sort | sed -n '1,160p'

Did you complete the Demo P build

Copy

Short Answer

Yes — I completed the first code-level Demo P build in src/Code.gs, but I did not create the packaged scripts/ production script file or update Current_Production/.

So the status is:

Item Status

Demo P logic in src/Code.gs Completed first pass

Ongoing sheet name Demo P instead of monthly Demo P name Implemented

Universal metadata columns on Demo P Implemented

PMR-level source hash helpers Implemented

Demo P template/header defaults updated Implemented

Apps Script runtime/integration test Not run

New scripts/vX.X.X_Production_Script.txt file Not created

New/updated Current_Production/ file Not created

What Was Completed

1. Demo P is now built as an ongoing sheet

The Demo P processing target is now the stable DEMO_P_PREFIX, meaning the workflow is aimed at the ongoing Demo P sheet rather than a monthly Demo P mm.yy sheet.

The final rename step also keeps the sheet name as Demo P.

2. Demo P now has universal metadata columns

I added universal metadata columns alongside the existing Demo P update columns:

Update Status

Update Month

Source Sheet

Columns Updated

Last Updated At

Source Hash

Previous Source Hash

Source Workflow

These were added to column formatting/configuration.

They were also added to the Demo P optional/output headers.

3. Demo P processing now writes update metadata and source hashes

During Demo P processing, the workflow now populates the legacy Demo P update fields and the new universal metadata/hash fields before writing processed Demo P data back to the sheet.

When Demo P is created from Raw Data, it also populates those universal metadata columns before the output sheet is created and then runs the processing pass with the Raw Data sheet as the source.

4. PMR-level source hash helpers were added

I added helpers to build source hashes by PMR, exclude volatile metadata fields, normalize dates/text, and compute a stable MD5 hash.

This is the foundation for the later metadata/hash-driven Monthly Change path.

5. Dashboard/default Demo P headers were updated

The default dashboard header set for Demo P now includes the universal metadata fields so rebuilt Demo P templates can include them.

What Is Not Yet Complete

1. I did not create the final production script artifact

I did not create a new file under scripts/, such as a new v1.5.xx_Production_Script.txt.

2. I did not update Current_Production/

I did not create or replace the current production script file in Current_Production/.

3. I did not run Apps Script runtime testing

I only validated JavaScript syntax with Node. I did not run this inside Google Apps Script, so the build still needs Apps Script workflow validation.

4. I should clean the line-ending diff before continuing

The latest commit does implement the Demo P build, but because src/Code.gs has mixed/CRLF line endings, the diff became noisier than ideal. Before building the next section, I should clean that up so the next PR/diff is easier to review.

Commands Used To Confirm

✓ `git show --stat --oneline --name-only HEAD`

✓ `git status -sb`

✓ `nl -ba src/Code.gs | sed -n`

'250,268p;3184,3205p;3438,3475p;3542,3556p;3628,3638p;3856,3886p;3917,4050p;14255,14275p'

Bottom Line

Demo P build: first pass complete in src/Code.gs.

Production packaging: not done yet.

Runtime validation: not done yet.

We have been saving them as scripts/v1.5.55_Rebuild_MENU_FUNCTIONS.txt.

Copy

Summary

Corrected the Demo P build location to match the rebuild-section convention by adding scripts/v1.5.55_Rebuild_DEMO_P_FUNCTIONS.txt instead of leaving the work only in active src/Code.gs. The new section header identifies it as the DEMO P FUNCTIONS rebuild candidate and documents its dependencies/purpose.

Reverted the prior active src/Code.gs Demo P edits from the last commit so this Demo P work now lives in the rebuild artifact path you described.

Added the Demo P header/metadata column definitions in the rebuild section, including the universal metadata fields: Update Status, Update Month, Source Sheet, Columns Updated, Last Updated At, Source Hash, Previous Source Hash, and Source Workflow.

Included the Demo P build/update workflow in the rebuild section, including keeping Demo P as the working source and routing updates through processDemoPAsWorkingSource_.

Added the universal metadata population and PMR-level source hash helpers to the Demo P rebuild file for the later metadata/hash-driven Monthly Change path.

Testing

✓ `node --check /tmp/demo_p.js`

✓ `git diff --check`

✓ `git status -sb`

Committed the correction as ed6d318 with title Move Demo P build to rebuild section, and created the PR record for that commit.

Files (2)

Clear all previous Diff files leaving only current files. Clear all caches, branch, containers.

Copy

Build next section

Copy

The generated diff exceeds our size limit and could not be extracted. This can happen when large binaries, minified assets, or an exceptionally extensive refactor are included in a single commit.

You are viewing a previous turn

Follow-ups are only available for the latest turn

View latest